

AI-Powered Legal Document Analysis System:
Clause Detection, Category-Level Risk Prioritization
and Plain-Language Explanation on CUAD

by

Jerin Aktar

22101279

Afnan Mazumdar

24141229

Shoyeb Hasan Sayem

22101386

A project submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
July 2026

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The project submitted is our own original work while completing degree at Brac University.
2. The project does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The project does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Jerin Aktar

22101279

Afnan Mazumdar

24141229

Shoyeb Hasan Sayem

22101386

Approval

The project titled “AI-Powered Legal Document Analysis System: Clause Detection, Category-Level Risk Prioritization and Plain-Language Explanation on CUAD” submitted by

1. Jerin Aktar (22101279)
2. Afnan Mazumdar (24141229)
3. Shoyeb Hasan Sayem (22101386)

of Summer, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering in July 2026.

Examining Committee:

Supervisor:
(Member)

Utsho Kumar Roy

Designation
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Sadia Hamid Kazi, PhD

Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Ethics Statement

This project uses the Contract Understanding Atticus Dataset (CUAD), a publicly released research dataset of 510 commercial contracts taken from public filings and annotated by legal experts under The Atticus Project. We did not collect or process any private, confidential, or personally sensitive documents, and no human subjects were involved. All experiments ran on our own hardware. The third-party models we fine-tuned (DistilBERT, FLAN-T5) and all supporting libraries are used under their open-source licenses, and every outside source is cited.

The system is a research prototype meant to assist contract review. It is not legal advice, and that is stated in the application itself. The ethical risks of relying on its output (false reassurance from missed clauses, over-reliance by non-lawyers, jurisdiction mismatch, the perspective from which risk is scored, the unreviewed authorship of the risk levels, and privacy in a real deployment) are discussed in detail in Section 3.4.

Abstract

This project builds and evaluates a three-model pipeline for automated contract review on CUAD (510 contracts, 41 clause categories), using a held-out contract-level 20% test split. A sliding-window DistilBERT presence classifier *loses* to a full-document TF-IDF baseline (micro-F1 0.694 against 0.775), and the gap survives training on all 53,662 windows, three seeds, six aggregation rules and a threshold sweep. A conjunctive ensemble reaches 0.776, but its margin is smaller than the bootstrap, seed and split variance alike. Span extraction reaches token-F1 0.763 given a correct window. The fine-tuned summarizer emits one of 41 self-written templates verbatim on every input, scoring 0.116 against clause-specific references. End to end only 21.7% of gold clauses survive all three stages, and the highest-scoring configuration misses 37.5% of High-risk clauses.

Keywords: legal NLP; CUAD; clause extraction; category-level risk prioritization; contract summarization; held-out evaluation

Acknowledgement

First of all, all praise to the Almighty for allowing us to complete this project without any major interruption. We are grateful to our supervisor, Utsho Kumar Roy, for his guidance and patience through every phase of this work, and especially for insisting on a strict held-out evaluation: that one requirement shaped how honestly this report presents its results. Finally, we thank our families for their constant support and encouragement. Without them, this work would not have been possible.

Table of Contents

Declaration	i
Approval	ii
Ethics Statement	iii
Abstract	iv
Acknowledgment	v
Table of Contents	vi
List of Figures	x
List of Tables	xi
Nomenclature	xii
1 Introduction	1
1.1 Background	1
1.1.1 System at a Glance	1
1.2 Rationale of the Study or Motivation	1
1.3 Problem Statement	2
1.3.1 Research Questions	3
1.4 Objective	3
1.5 Methodology in Brief	4
1.6 Scopes and Challenges	4
1.7 Report Organization	5
2 Literature Review	6
2.1 Preliminaries	6

2.1.1	Transformer Encoders and Model Distillation	6
2.1.2	Long-Document Strategies and Multiple-Instance Learning . .	6
2.1.3	Text-to-Text Models for Summarization	7
2.2	Review of Existing Research	8
2.2.1	Legal NLP and the CUAD Benchmark	8
2.2.2	Legal Risk Assessment and Decision Support	8
2.2.3	Plain-Language Legal Communication	9
2.2.4	Faithfulness and Hallucination in Generated Summaries . . .	10
2.3	Summary of Key Findings	10
3	Requirements, Impacts and Constraints	12
3.1	Final Specifications and Requirements	12
3.1.1	Functional requirements	12
3.1.2	Non-functional requirements	12
3.1.3	Data and resource requirements	13
3.1.4	Design constraints	13
3.2	Societal Impact	14
3.3	Environmental Impact	14
3.4	Ethical Issues	15
3.4.1	Data Handling and Privacy	16
3.5	Standards	16
3.6	Project Management Plan	17
3.6.1	Schedule	17
3.6.2	Budget	17
3.6.3	Resource management	17
3.7	Risk Management	18
3.8	Economic Analysis	18
4	Proposed Methodology	21
4.1	Design Process or Methodology Overview	21
4.2	Preliminary Design or Design (Model) Specification	22
4.2.1	Stage 2A: Presence Classification	22
4.2.2	Stage 2B: Span Extraction	25
4.2.3	Stage 1: Plain-Language Summarization	26
4.2.4	Category-Level Risk Prioritization	26

4.2.5	Leakage Discipline	29
4.3	Data Collection	30
4.3.1	Data Cleaning	30
4.3.2	Data Transformation	30
4.3.3	Data Integration	31
4.3.4	Data Reduction	31
4.3.5	Summary of Preprocessed Data	32
4.4	Implementation of Selected Design	33
4.4.1	Validation Protocol, and Where the Hyperparameters Came From	34
4.4.2	Optimization Schedule	36
4.4.3	Training the Presence Classifier	37
4.4.4	Training the Span Extractor	37
4.4.5	Training the Summarizer	38
4.4.6	Engineering Notes: Training on Consumer Hardware	38
4.4.7	Measured: What Sparse Attention Would Have Cost	39
4.4.8	End-to-End Data Flow	40
4.4.9	The Clausify Application	41
5	Result Analysis	47
5.1	Performance Evaluation	47
5.1.1	Presence Classification	47
5.1.2	Span Extraction	49
5.1.3	Summarization	51
5.2	Analysis of Design Solutions	53
5.2.1	End-to-End: What Actually Reaches the User	54
5.2.2	Risk-Weighted Evaluation: Does the Metric Match the Pur- pose?	56
5.2.3	What the errors actually look like	57
5.2.4	Does the AND rule punish rare categories harder than com- mon ones?	58
5.3	Final Design Adjustments	59
5.3.1	Ablation: was max-pooling the right aggregation rule?	59
5.3.2	The Threshold and What the Confidence Score Means	60
5.4	Statistical Analysis	62

5.4.1	How much of that ranking is real?	62
5.4.2	How much of this moves if we only change the random seed? .	63
5.4.3	Was the transformer simply undertrained?	64
5.4.4	A third source of variance: the split itself	65
5.5	Comparisons and Relationships	66
5.6	Discussions	66
5.6.1	Usability	67
5.6.2	Honest Limitations of the Demo	68
6	Conclusion	73
6.1	Summary of Findings	73
6.2	Contributions to the Field	74
6.3	Threats to Validity	75
6.3.1	Internal validity: could our own procedure have produced these numbers?	75
6.3.2	Construct validity: do our metrics measure what we claim? .	76
6.3.3	External validity, where do these results transfer?	77
6.3.4	Statistical conclusion validity: are the intervals trustworthy? .	77
6.4	Limitations	77
6.5	Recommendations for Future Work	79
	Bibliography	81
	A Full Risk Taxonomy	83
	B Reproducibility Checklist	85
	C Repository and Artifact Layout	88

List of Figures

1.1	Three questions the system answers	2
4.1	System pipeline	21
4.2	Retrieval ceiling	23
4.3	Sliding-window generation	24
4.4	Multiple-instance max-pooling	24
4.5	Ensemble rules	25
4.6	Presence vs. focus window	26
4.7	Risk-level distribution	29
4.8	Per-category split balance	33
4.9	Learning-rate schedules	36
4.10	Presence classifier loss	37
4.11	Summarizer loss	38
4.12	Pipeline data flow	40
4.13	Clausify input screen	42
4.14	Highlighted contract and checklist	43
4.15	Clausify results view	44
4.16	High-risk clause cards	45
5.1	Presence model micro-F1	48
5.2	Confusion matrix heatmap	50
5.3	Top-12 per-category F1	51
5.4	Threshold and calibration	61

List of Tables

2.1	Related work	11
3.1	Resource requirements	13
3.2	Project schedule	18
3.3	Risk management plan	19
3.4	Cost–benefit comparison	20
4.1	Retrieval hit-rates	23
4.2	Train/test split	33
4.3	Training configurations	34
4.4	Validation arrangements	35
4.5	Longformer vs. DistilBERT cost	39
4.6	Application latency	46
5.1	Presence model comparison	48
5.2	AND-ensemble confusion matrix	49
5.3	Confusion-matrix metrics	49
5.4	Categories with zero F1	49
5.5	Span extraction results	50
5.6	Summarizer ROUGE-L	52
5.7	Fine-tuned vs. zero-shot output	53
5.8	End-to-end clause survival	54
5.9	Span-stage loss breakdown	55
5.10	Results by risk band	56
5.11	Representative errors	70
5.12	Rare vs. common categories	70
5.13	Aggregation ablation	71
5.14	Cluster bootstrap results	71

5.15	Seed variance	71
5.16	Training-budget control	71
5.17	Split variance	72
5.18	Final test results	72
A.1	Full risk taxonomy	83

Nomenclature

The next list describes several symbols & abbreviation that will be later used within the body of the document

CUAD	Contract Understanding Atticus Dataset
EM	Exact Match
FN / FP	False Negative / False Positive
IP	Intellectual Property
LR	Logistic Regression
MIL	Multiple-Instance Learning
MPS	Metal Performance Shaders (Apple GPU backend)
NLP	Natural Language Processing
QA	Question Answering
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
SQuAD	Stanford Question Answering Dataset
TF-IDF	Term Frequency-Inverse Document Frequency
TN / TP	True Negative / True Positive
BERT	Bidirectional Encoder Representations from Transformers
ECE	Expected Calibration Error

Glossary of terms

Clause category One of the 41 types of contractual provision annotated in CUAD (e.g., *Governing Law*, *Non-Compete*, *Cap on Liability*).

Contract-level split A train/test partition that assigns whole contracts to one side, so no clause from a training contract can appear in the test set.

Presence classification Deciding, for each clause category, whether a contract contains at least one clause of that category.

Span extraction Locating the exact character span of a clause inside the contract text, formulated as extractive question answering.

Max-pooling (over windows) Scoring every sliding window of a document and taking the maximum window score as the document-level prediction.

Retrieval ceiling The upper bound that a retrieval step imposes on a downstream model’s recall: clauses in windows the retriever never selects can never be found.

Weaker party The contract signer with less bargaining power, from whose perspective the risk taxonomy is scored.

Chapter 1

Introduction

1.1 Background

Contracts are part of almost everything people and businesses do, but reading them takes a lot of time. A typical contract in our dataset is about 33,000 characters long (roughly 5,000 words), and the longest is more than 338,000 characters. Somewhere in all that text are the parts that really matter: who owns new intellectual property, whether there is a limit on how much you can owe, whether the contract renews on its own, and whether you are stopped from working with competitors after it ends. Having a lawyer read a contract takes hours and costs money that many people and small businesses cannot afford. So people often sign contracts they do not fully understand and take on risks they never noticed.

The Contract Understanding Atticus Dataset (CUAD) [1] was created to help automate this kind of review. It has 510 real business contracts in which legal experts marked every clause that belongs to one of 41 important categories. This marking work is said to be worth about two million US dollars of lawyers' time. However, CUAD only treats the problem as finding text: given a category, find the clause. Once it finds a clause such as "Uncapped Liability", it stops. It does not warn the reader what the clause means for them, it does not explain it in simple words, and it does not check how clauses affect each other.

1.1.1 System at a Glance

Chapter 4 explains the system in detail. Figure 1.1 shows the whole system in one picture. The system answers three questions about a contract, in order: *what is in it*, *where is it*, and *what does it mean*. CUAD was built to test only the first two. The third step, shaded in the figure, is what this project adds.

1.2 Rationale of the Study or Motivation

Our project tries to close the gap between finding a clause and understanding it. On top of finding clauses, we add two things. First, a simple risk table that gives



Figure 1.1: Three questions the system answers

every clause *category* a High, Medium or Low level, judged from the side of the weaker party. Second, a text model that is meant to turn each clause into one plain-English sentence. We built the clause-finding models and the risk table into *Clausify*, a prototype web app: a user pastes or uploads a contract and gets back a list of clauses sorted by risk. Both added parts worked less well than we hoped. The risk table has not been checked by a lawyer (Section 4.2.4), and the text model learned to repeat a fixed sentence for each category instead of describing the actual clause (Section 5.1.3). Chapters 5 and 6 report these results honestly.

So the reason for this study is practical as well as academic. Earlier work on CUAD answers the question “where is the clause?” for a lawyer who already knows what to look for. The people who need help most (individuals, freelancers and small businesses who sign contracts without a lawyer) also need to know which clauses matter most to them and what each one means in everyday language. A system that gives those answers, and is honest about how often it is wrong, could make contract review possible for people who cannot pay for it today. Just as important, we wanted to measure how well such a system really works instead of assuming it. A contract-review tool that looks confident but quietly misses clauses is worse than having no tool at all.

1.3 Problem Statement

In short, the project works on four connected problems:

1. **Finding which clauses are present.** For each of the 41 CUAD categories, decide whether a contract has at least one clause of that type. The main difficulty is length: a typical contract is about $16\times$ longer than the amount of text a standard transformer model can read at once.
2. **Finding the exact clause text.** For each category that is present, find the exact words of the clause in the contract. We treat this as a question-answering task, in the same style as SQuAD [2].
3. **Explaining in plain language.** Rewrite each clause as one sentence that a non-lawyer can understand.
4. **Showing the risk.** Give every clause found a risk level and a one-line reason, so the result reads like a ranked list of warnings instead of a list of quotes. We did not test whether this actually helps users (Section 6.3).

1.3.1 Research Questions

Our objectives turn into five research questions, which the rest of this report answers. We give the answers here so readers can check each claim against the evidence instead of searching for it.

RQ1 *Can a transformer that reads a contract in small windows beat a simple word-based model that reads the whole contract, at finding which clauses are present?*

No. The simple model wins by +0.081 micro-F1. The gap stays even when the transformer is trained on all its training data (Section 5.4.3) and with three different random seeds (Section 5.4.2). Answered in Chapter 5.

RQ2 *How well can the system find the exact clause text in long contracts?*

Token-F1 is 0.763 (95% CI [0.740, 0.782]) *when the model is given a window that contains the clause*, but only 0.386 when the window is chosen by the first model (Sections 5.1.2 and 5.2.1).

RQ3 *Does the summarizer really describe each specific clause in plain English?*

No. For every input it copies one of 41 sentences we wrote in advance. Against clause-specific reference sentences, it scores no better than just printing that fixed sentence without any model (Section 5.1.3).

RQ4 *How do errors add up across the whole system?*

They multiply, and it is worse than the separate scores suggest: only 21.7% of real clauses get through all three steps, compared with the $\approx 64\%$ that the separate scores would predict (Section 5.2.1).

RQ5 *Do the system’s mistakes avoid the clauses that would hurt a signer most?*

No, the opposite. The setup we report as our main result misses 37.5% of High-risk clauses, while the model we call weaker misses only 15.9% (Section 5.2.2). The overall score and the harm to the user point in different directions.

Three of these five answers are negative. We report them as real results of the project, not as failures. Each one is something a reader would have got wrong by looking only at the separate model scores.

1.4 Objective

- Build and fairly test a complete three-model CUAD system on an ordinary laptop, using an 80/20 split by contract so that every reported number comes from contracts the models have never seen.
- Check whether a transformer really beats a strong simple model at finding which clauses are present, instead of assuming it does.
- Measure how well the exact clause text is found, using token-level F1 and exact match on the test set.

- Test the summarizer against two different sets of reference sentences, because one automatic score turned out to measure the reference sentences rather than the task.
- Measure how many clauses survive the whole system from start to finish, not only the score of each step.
- Put the trained models into a working demo app that shows clauses sorted by risk.

1.5 Methodology in Brief

We tested every design before building on it, using one fixed 80/20 split of CUAD by contract (408 training contracts and 102 test contracts). After exploring the data (Section 4.3.5), we first built a simple word-based model (TF-IDF with logistic regression) that reads the whole contract, *before* training any transformer. We also measured the highest recall a “search first” design could ever reach (Section 4.2.1). Based on that, we trained a DistilBERT model that scores the contract in small windows and takes the highest window score as the answer for the whole contract (multiple-instance learning). We combined it with the simple model using an AND rule, which reports a clause only when both models agree. A DistilBERT question-answering model finds the exact clause text, a fine-tuned FLAN-T5-small model writes a one-sentence plain-English explanation, and a hand-made table gives each of the 41 categories a High, Medium or Low risk level.

Every model was tested once on the 102 test contracts. We report each score together with confidence intervals from a bootstrap over contracts, and we checked how the results change with the amount of training data, the random seed, the way window scores are combined, the decision threshold and the train/test split. We also tested the full system from start to finish. Finally, we put the trained models into *Clausify*, a Streamlit web app that gives a risk-sorted clause report for any contract. Chapter 4 describes the method in full.

1.6 Scopes and Challenges

Scope. The project covers English business contracts like those in CUAD: 510 contracts from US SEC filings, marked for 41 clause categories. Within that scope it handles four tasks (finding which clauses are present, finding their exact text, explaining them in plain language and showing their risk level) and delivers a working prototype app. The following are outside the scope on purpose: legal advice of any kind; contracts in other languages or countries (including Bangladeshi contracts, which inspired the work but were not tested); judging the risk of a clause’s exact wording; finding conflicts between clauses; and a formal user study of the app. All models were sized so they can be trained and run on one ordinary laptop.

Challenges. Six challenges shaped the design:

1. **Very long documents.** A typical contract is about 33,000 characters. That

is roughly $16\times$ the 512-token limit of a standard transformer (about 2,000 characters), and the longest contract is more than 338,000 characters, about $170\times$.

2. **Very uneven categories.** Some categories appear in 100% of contracts and some in only 2.5%; a few have only a handful of examples in the whole dataset.
3. **A small, fixed dataset.** The 510 contracts are all there is, and expert marking is too costly to add more, so we had to be careful about over-fitting and about test data leaking into training.
4. **Limited hardware.** Training ran on an Apple-Silicon laptop without a CUDA GPU. Our first-choice model (DeBERTa-v3) was unstable on the MPS backend, and GPU memory was not freed between training runs.
5. **No plain-language answers to learn from.** CUAD has no plain-English versions of clauses, so we had to write our own reference sentences, and this limited what the summarizer could learn.
6. **No agreed answer for risk either.** Contract risk depends on which side you are on, the situation and the country, and there is no expert-checked risk model for CUAD’s 41 categories.

1.7 Report Organization

The rest of this report is organized as follows. Chapter 2 reviews background and earlier work. Chapter 3 sets out the system’s requirements, its effect on society, the environment, ethics and cost, and the project plan. Chapter 4 explains the method: how we designed the system, which other designs we considered, how the data was prepared, and how the models and the Clausify app were built. Chapter 5 analyses the results on the test set, including the statistics and the changes we made because of them. Chapter 6 sums up the findings, contributions and limits, and suggests future work. The appendices contain the full 41-category risk table, a checklist for repeating the work and the layout of the project files.

Chapter 2

Literature Review

2.1 Preliminaries

This section explains the models and methods the project is built on.

2.1.1 Transformer Encoders and Model Distillation

The transformer [3] is a type of neural network that reads text using self-attention instead of reading it word by word. BERT [4] showed a simple recipe that this project relies on: first train a model on a very large amount of general text, then fine-tune it for a specific task using a much smaller amount of labelled data. This matters a lot for us, because 510 contracts is a small dataset for deep learning.

Because all our work runs on one laptop, we use DistilBERT [5]. It is a smaller, six-layer version of BERT that keeps about 97% of BERT’s language ability while being 40% smaller and 60% faster. The published CUAD results use much larger models, so we give up some accuracy in exchange for being able to run everything on normal hardware. We point out this size difference whenever we discuss our results.

2.1.2 Long-Document Strategies and Multiple-Instance Learning

Contracts are much longer than a transformer can read at once. There are three usual ways to deal with this: (i) *retrieval*, where a quick, cheap method picks the few parts of the text most likely to contain the answer and only those are given to the transformer; (ii) *sliding windows*, where the model reads every part of the text and the scores are combined into one answer for the whole document; and (iii) *sparse-attention models* such as Longformer, which can read a longer piece of text at once.

The third option is often dismissed in one line as “too expensive”, but that alone is not a good reason. We did not run a Longformer experiment, so what follows is a design choice, and we call it that. The one part of it we could check cheaply on our own laptop, the cost, we did check, and Section 4.4.7 reports the result.

We had three reasons, and only the third is about cost. **First, and most important, it would not solve the problem.** Longformer can read 4,096 tokens at once. At about four characters per token, our typical contract of 33,143 characters is about 8,300 tokens and the longest is about 85,000. So Longformer would still see only half of a typical contract and one-twentieth of the longest one. It makes the window eight times wider, but that is still not wide enough for this data. We would still need windows and a way to combine their scores, which is the design we ended up using anyway, just with a much more expensive model underneath. This reason is simple arithmetic about the dataset rather than an experiment, but the arithmetic is clear: CUAD contracts are about ten times too long for a wider window to fix.

Second, availability. Our whole system uses small (distilled) models so that it can be trained and run on one laptop. We do not know of a small version of Longformer that is as reliable and well supported as DistilBERT, so using Longformer would have meant giving up the small models that make the rest of the project easy to repeat.

Third, cost in practice. We believed that fine-tuning a Longformer with 148.7M parameters at 4,096 tokens was not possible on one Apple-Silicon laptop. We were also worried about special attention code on a non-CUDA machine, after DeBERTa-v3 produced NaN (broken) values on the MPS backend (Section 4.4.6). When we later measured it (Section 4.4.7), we found the memory part of this belief was *wrong*: one training step does fit at batch size 1. The time part, however, was right by a very large margin. We kept our original reasoning in the text and reported the correction next to the measurement, instead of quietly changing the reasoning to match the result. There is also a cost when the app runs: *Clausify* scores every window of an uploaded contract for all 41 categories while the user waits, so it is only usable if this takes seconds. This can be measured without any training, so we measured it: Section 4.4.7 gives the speed and the peak memory of Longformer-base at 4,096 tokens on our laptop, next to the DistilBERT setup we actually used.

On a CUDA computer with more memory, the third reason would be weaker and the second might disappear, but the first would still hold. So this is a design choice backed by one measurement and some arithmetic. It does not prove that sparse attention cannot work on CUAD. A properly trained Longformer with windows would be a fair comparison to ask for, and we list it as future work.

Reading a document in windows and taking the highest window score is known as *multiple-instance learning* (MIL). The document is treated as a bag of windows, and the bag counts as positive if any one window is positive. Taking the maximum score does exactly this. MIL has a well-known weakness, which our experiments confirmed: it produces too many false alarms. A document has dozens of windows, so the chance that *some* window fires by mistake grows with the length of the document, and precision drops.

2.1.3 Text-to-Text Models for Summarization

T5 [6] treats every language task as “text in, text out”, which suits clause simplification well: legal text goes in and a simple sentence comes out. FLAN-T5 [7] is a

version of T5 that was trained on many tasks with instructions, so it already gives useful answers to a simple prompt such as “summarize in plain English:” without extra training. We score summaries with ROUGE-L [8], which measures how many words the output shares, in the same order, with a reference sentence. Chapter 5 shows why this score must be checked by actually reading the outputs when there are only a few reference sentences.

2.2 Review of Existing Research

This section reviews earlier work on legal language processing, legal risk, plain-language legal writing, and the accuracy of machine-written summaries.

2.2.1 Legal NLP and the CUAD Benchmark

People have long wanted to use language technology on legal text, but progress was slow for a simple reason: labelling legal text needs lawyers, and lawyers are expensive. CUAD [1], released by The Atticus Project [9] at the NeurIPS 2021 Datasets and Benchmarks track, changed this. It provides 510 real business contracts from public SEC filings, in which experienced lawyers marked every piece of text that belongs to one of 41 important clause categories, such as governing law, non-compete, limits on liability and IP assignment. The data uses the SQuAD format [2]: each (contract, category) pair is asked as a question about the contract text, and the marked clauses are the answers, with their exact character positions.

The original CUAD paper treats the task as question answering and finds that bigger models do much better. Its best results come from the extra-large DeBERTa models, and even those scores are modest, which shows how hard it is to find legal clauses. Two features of CUAD affect any model built on it. The first is that categories are very uneven: some appear in almost every contract, while others appear only about a dozen times in the whole dataset. The second is length. Contracts are far longer than a transformer can read at once, so every practical system first has to answer one question: how can a model that reads 512 tokens at a time read a 50,000-character contract?

2.2.2 Legal Risk Assessment and Decision Support

The risk part of this project has the least earlier work behind it, and that is a weakness of our work rather than of the field. Since we tell a non-lawyer which clauses to worry about, the obvious question is what this advice is based on. The honest answer is that we did not use an existing framework. We wrote the risk table ourselves (Section 4.2.4), and that is why we call it a prototype.

Earlier work does explain why this is hard. Legal language benchmarks usually treat “risk” indirectly, by sorting text into legally meaningful categories rather than judging how serious something is. LexGLUE [10] collects tasks such as predicting court decisions and classifying laws, and LEGAL-BERT [11] shows that models pre-trained on legal text do better on them. But none of these tasks asks how *dangerous*

a clause is for a particular person signing it. CUAD itself [1] says its categories are the clauses lawyers look for when checking a deal, which shows they are important without saying how important.

So how serious a clause is remains an open question, and it is hard to learn from data alone. First, risk depends on which side you are on: the same unlimited indemnity is dangerous for the party giving it and useful for the party receiving it. A risk label only makes sense once you decide whose side you are on, a problem Section 4.2.4 handles only by making an assumption. Second, risk depends on the situation: a \$50,000 limit on liability means little to a large company but can ruin a one-person business. Third, risk depends on the country: a non-compete that is binding in one country may be invalid in another. Our table gives one fixed level per category, so on purpose it ignores all three.

This has two effects on how our work should be read. First, our risk table should be compared with expert opinion, not with a published risk model, since we know of no published model that covers CUAD’s 41 categories. That expert check, as Section 4.2.4 explains, has not been done yet. Second, work on explainable legal AI argues that tools in this area should show their reasoning instead of just giving a score. Our table does this well: every level comes with a written reason that can be questioned line by line (Appendix A). This openness is the real strength of the design, but being open is not the same as being correct.

2.2.3 Plain-Language Legal Communication

Chapter 5 finds that our summarizer does not really summarize, and the relevant earlier work is about *simplifying* text, not about T5. The difference matters. Summarizing makes text shorter, while simplifying says the same thing in easier words. For a contract clause, making it shorter is often the wrong goal, because the conditions and exceptions that a summary would drop are often exactly what decides what the clause does.

The closest earlier work is Manor and Li [12], who collected plain-English summaries of contract sections and found that models trained on them tend to give general, repeated answers. This is an early version of the problem our template test measures. Measuring how easy a text is to read is a much older field [13], and it offers something our evaluation does not have: a way to check that the output really is easier to read, not just shorter. We do not report any readability score, which, looking back, is a gap, because the whole point of this part is to make clauses easier to understand.

The main lesson from this work, which our results also show, is that scores based on reference sentences measure the reference sentences. With only 41 different target sentences, ROUGE-L rewards the model for recognising the category. Section 5.1.3 measures this directly by changing the reference sentences while keeping everything else the same.

2.2.4 Faithfulness and Hallucination in Generated Summaries

When a system shows machine-written sentences next to a user’s own contract, it is a safety issue whether those sentences are true, not just a matter of quality. Research on summarization has shown that fluent machine-written summaries often say things that are not in the source. Maynez et al. [14] find that many generated summaries contain content the input does not support, and that ROUGE agrees poorly with human judgements of accuracy. Kryściński et al. [15] suggest checking factual accuracy as a separate task. Scores such as BERTScore [16] measure meaning better than simple word overlap, but they do not check facts either.

This affects our system in an unusual way. A model that only outputs fixed template sentences **cannot make things up in the usual sense**: it can only produce one of 41 sentences written by people, and each is a correct description of its category. Instead, it fails in the opposite way: it says nothing about the specific clause, and when it picks the wrong template (30% of the time, Section 5.1.3) the output is fluent and confident but about a completely different kind of clause. For a legal assistant this is arguably *worse* than a made-up detail, because it cannot be spotted from the output alone. The sentence is well written and correct in itself; only comparing it with the clause shows that it does not describe it.

It also means our evaluation is missing the checks this research recommends. We report ROUGE-L against two sets of reference sentences and we read outputs by hand. We do not report BERTScore, an automatic fact check, or a human judgement of whether the legal meaning is kept. For a part that non-lawyers will see, the last one matters most. Section 6.5 lists it as future work.

2.3 Summary of Key Findings

Earlier CUAD work stops once the clause is found. A person signing a contract probably needs more: which clauses matter, how much, and what they mean in plain language. This project tries to add that layer (a prototype risk table for each category, plain-English template sentences and a demo app). It also re-tests, on a strict test set, the common belief that a transformer must beat simple word-based methods at deciding which clauses a contract contains. We should say from the start that, of the three added parts, the results support only the demo app as something that works. The risk table has not been checked by an expert, and the plain-English part outputs category templates rather than summaries, so neither is a solved problem. What this added layer mainly gives is a measurement of how far it is from working.

Table 2.1 compares this work with the earlier work above. Two differences stand out. The obvious one is scope: earlier work stops at finding the clause text, while our output is a risk-sorted, plain-English report, which is the part a non-lawyer can act on. The less obvious one is in the “Long inputs” column. Every system on CUAD must choose how to handle long documents, but usually that choice is just stated. Here we measured it before any training: the 64.0% retrieval limit in

Section 4.2.1 is what ruled out retrieval and led us to multiple-instance learning. The Model column also shows a cost: our small models (67M parameters) are about ten times smaller than the DeBERTa-xlarge model behind CUAD’s best published result, so our extraction scores cannot be directly compared with theirs, and we do not claim they can.

Table 2.1: Related work

Work (data)	Task	Model	Long inputs	Risk	Plain English	Evaluation
Hendrycks et al. [1] CUAD, 510 contracts	Span extraction	DeBERTa-xlarge	Full-document QA with a large model	No	No	Held-out; precision at 80% recall
Chalkidis et al. [11] several legal corpora	Domain pre-training and classification	LEGAL-BERT	Truncation to 512 tokens	No	No	Per-task held-out
Chalkidis et al. [10] LexGLUE, 7 tasks	Legal language understanding	BERT and RoBERTa models	Depends on the task	Indirect (task labels)	No	Multi-task benchmark
Manor and Li [12] TL;DR-Legal style	Plain-English summarization	Seq2seq	Section-level inputs	No	Yes	ROUGE and manual review
Beltagy et al. [17] long-document benchmarks	New architecture	Longformer	Sparse attention, 4,096 tokens	No	No	Task benchmarks
Retrieval-QA pipelines various	Span extraction	Various	Retrieval then reader (ceiling usually assumed)	No	No	Various
This project CUAD, 510 contracts	Presence, span, category risk and template text	DistilBERT and FLAN-T5 (small)	Sliding-window MIL; ceiling <i>measured</i> , sparse attention <i>benchmarked</i>	Yes (category level, prototype)	Yes (template based)	Contract-level held-out; cluster bootstrap; seed, budget and split controls; end-to-end

Note. “Long inputs” is how each work handles documents longer than the encoder window; “Evaluation” is the level at which results are reported.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

3.1.1 Functional requirements

- FR1** The system shall accept a contract as pasted text or as an uploaded `.pdf`, `.docx` or `.txt` file.
- FR2** For each of the 41 CUAD clause categories, the system shall decide whether the contract has a clause of that type and show a score.
- FR3** For each category found, the system shall show the clause text from the user's own contract.
- FR4** The system shall give every clause found a High, Medium or Low risk level and a one-line reason in plain English.
- FR5** The system shall show the clauses grouped by risk level (High first) and sorted by score within each group.
- FR6** The system shall let the user analyze either a whole contract or a list of separate clauses, and shall show the score of all 41 categories.

3.1.2 Non-functional requirements

- **Privacy.** All models shall run on the user's own computer; no contract text shall be sent to another service or saved to disk.
- **Hardware.** All models shall train and run on one ordinary laptop without a CUDA GPU.
- **Speed.** Analyzing a typical contract shall take no more than tens of seconds on a CPU (measured: median 27.1 s, worst case 32.4 s; Table 4.6).

- **Fair testing.** Every reported score shall come from contracts the models never saw during training (split by contract), and no setting shall be chosen by looking at the test set.
- **Repeatability.** The split, the baseline model and all trained models shall be saved, and a separate notebook shall recreate every reported number from them.
- **Openness.** The app shall say that it is not legal advice, and every risk level shall come with a written reason (Appendix A).

3.1.3 Data and resource requirements

Table 3.1 lists the data, hardware and software the project uses. All of it is either free to use or was already owned by the team.

Table 3.1: Resource requirements

Resource	Specification
Dataset	CUAD v1 (The Atticus Project), 510 annotated contracts: <code>master_clauses.csv</code> and <code>CUAD_v1.json</code> , obtained from the Hugging Face hub
Hardware	Apple M4 laptop, 16 GB unified memory, MPS (Metal) GPU backend; no CUDA GPU
Language and runtime	Python 3.13.5
Deep learning	PyTorch 2.8.0, Hugging Face <code>transformers</code> 5.12.1, <code>datasets</code> 5.0.0
Classical ML and data	scikit-learn 1.6.1, NumPy 2.1.3, pandas 2.2.3, <code>rouge_score</code>
Pre-trained models	<code>distilbert-base-uncased</code> (presence and span), <code>google/flan-t5-small</code> (summarizer)
Application	Streamlit (≥ 1.40) single-page web application
Development tools	Jupyter notebooks, Git and GitHub, $\LaTeX$

3.1.4 Design constraints

The requirements above meet four limits that shaped the final design. **Computing power:** one laptop cannot train very large models such as the DeBERTa-xlarge model used in the original CUAD work, so we chose smaller (distilled) models. **Data:** the 510 contracts cannot be extended, so we had to start from pre-trained models and split the data by contract. **Speed:** the app must score every window of a contract for all 41 categories, which is why it reads at most 30 windows. **Privacy:** the models must run locally, so online language-model services could not be used at all.

3.2 Societal Impact

Access to legal help. Having a lawyer review a contract takes hours and costs money that many people and small businesses cannot afford, so many people sign contracts they do not fully understand. A tool that points a non-lawyer to the clauses most likely to hurt them (unlimited liability, IP assignment, non-competes, automatic renewal) can make this easier and help people ask the right questions before signing. The main users we have in mind are freelancers, start-ups and small businesses, the groups least able to pay for professional review.

Health and safety. The system has no direct effect on physical health or safety. Its indirect effect is on money and stress: signing a contract without understanding it can lead to disputes, losses and worry, which better-informed signing could reduce.

Legal effects. The system is not legal advice, and it says so. Its biggest risk to society is *false reassurance*: a missed clause gives no warning, and no warning can be read as “nothing to worry about”. Our measurements show this risk is real. Only 22.0% of real clauses get through the whole system (Section 5.2.1), so the tool must be presented as a help for reading a contract, never as a replacement for a lawyer.

Culture and region. CUAD contains US business contracts written in English. Risk levels that make sense under US law may not apply in other countries, and contracts are written differently in different places. The work was inspired by the situation in Bangladesh, but no Bangladeshi contract was tested, so we cannot yet claim these benefits there.

3.3 Environmental Impact

The project was designed to need little computing power. All models are small (DistilBERT with 67M parameters and FLAN-T5-small). Training the three models used in the app took about 1.5 hours on a laptop, and the app runs on a CPU using at most about 1.5 GB of memory (Table 4.6).

Including the extra test runs (the full-data presence run of 140.7 minutes, two extra seeds, the evaluation runs and the Longformer test), the whole project used about ten hours of laptop computing. If a laptop uses 20–30 W under load (an estimate, not a measurement), this is well under one kilowatt-hour of electricity. That is very small compared with the many-GPU training often used for large legal language models.

Two design choices also keep the running cost low. The models run locally, so no data centre is used for each contract, and the system avoids large text-generating models, which use far more energy per request. The price is accuracy: Section 4.4.7 shows that larger models were rejected partly because of cost, and Chapter 5 reports what the small models achieve.

3.4 Ethical Issues

The Ethics Statement at the front of this report explains how the data and models were obtained. This section looks at the ethical issues that come from *using* the system, and the duties these place on the people who build it.

The system is a research prototype meant to help with contract review. It is not legal advice, and the app says so in a notice shown above every analysis. For a tool like this, the more serious ethical questions are not about the data but about what happens when someone relies on its output, so we list them clearly here.

False reassurance is the main danger. A clause the system does not find gives no warning, and no warning is easily read as “there is nothing to worry about”. Our measurements show this reading would be wrong: 21.6% of real clauses are missed at the first step, only 22.0% get through the whole system (Section 5.2.1), and the setup we report as our main result misses 36.9% of the High-risk clauses (Section 5.2.2). A user who checks a contract with this tool and sees three warnings cannot assume there are only three problems. Because missed clauses are silent while false alarms are visible, we believe the share of High-risk clauses found is the measure that matters most ethically, even though it is not the one the overall score rewards.

Too much trust by non-lawyers. The app is made for people without legal training, and they are the people least able to notice when it is wrong. Plain-English text shown with a confidence bar looks more certain than the model really is, and Section 5.3.2 shows the bar is not even a true probability: scores above 0.9 mean only about a 73% chance that the clause is really there. The plain-English reason on each card is a fixed sentence for the whole category from the risk table, not a reading of the user’s clause. The summarizer, which was meant to describe each clause, also turned out to output one fixed sentence per category (Section 5.1.3). The app labels the summarizer’s sentence clearly and warns when it belongs to a different category than the card. Shown without these warnings, both the bar and the reason would invite more trust than the evidence supports.

Different laws in different countries. CUAD follows US business law. Risk levels that are right there may be wrong elsewhere: a non-compete that is binding in one country may be invalid in another, and then a High badge would be misleading. The work was inspired by the situation in Bangladesh and we tested no Bangladeshi contract, so this is a real limitation, not just a possible one.

Whose risk is measured. The risk table assumes the user is the side with less bargaining power (Section 4.2.4). For the side that wrote the contract, the same badges are wrong in the other direction: a clause marked High for the signer may be exactly what the writer wanted. The system cannot tell which side the user is on.

Who wrote the risk levels. The risk levels were written by three computer science students and have not been checked by anyone with legal training (Section 4.2.4). Showing them in a polished app makes them look more official than they are. We believe an expert check must be done before any use beyond a demonstration.

Privacy in real use. Our experiments used only public contracts, so privacy was

not an issue here. A real deployment would face it at once, because contracts are among the most sensitive documents a business has. Section 3.4.1 explains what the demo app does and does not do with an uploaded contract, and what a real deployment would need to add.

3.4.1 Data Handling and Privacy

The app accepts contract text, and contracts are among the most sensitive documents a business has. Our experiments used only public CUAD contracts, so privacy was not an issue during this research. But the demo invites users to paste their own contracts, so we state exactly what happens to them.

What the demo does. All the models run locally: the DistilBERT models, the TF-IDF model and the FLAN-T5 summarizer are loaded from disk and run in the same program as the app, so no contract text leaves the computer the app runs on. An uploaded `.pdf`, `.docx` or `.txt` file is read into memory and turned into text on that computer. Nothing is saved to disk, no database or log keeps the text, and no usage data is collected. When the browser session ends the text is gone: it exists only in Streamlit’s memory while the page is open. The report the user can download is created on request and saved only where the user chooses.

What this depends on. This is only true because the demo runs on the user’s own computer. On a shared server, the same code would put the user’s contract in the memory of a machine they do not control, and Streamlit alone would not stop the server’s owner from reading it. The privacy comes from where the app runs, not from anything the app itself does, and we should not claim otherwise.

What is missing on purpose, and would be needed for real use. There is no login, so anyone who can reach the computer can open the app. There is no file-size limit beyond Streamlit’s default and no time limit, so a very large upload can keep the app busy. PDF and Word files are read with standard open-source libraries (`pypdf` and `python-docx`); a real deployment would need to treat these files as untrusted input, because a harmful file could try to attack the program that reads it. There is no rule for keeping data, because no data is kept. Party names are not removed before processing, which would matter if any part were ever moved to an online model service.

Our rule for any future change. As soon as any part of the system calls an online service, privacy changes completely: the contract would be sent to someone else and might be kept by them. The current design avoids this completely by using small local models. We see this as worth keeping, not just a side effect of using a laptop: it is one of the few places where the limits we worked under led to a better design.

3.5 Standards

No formal engineering standard applies to a research prototype like this, but the project followed several common practices:

- **Data format.** CUAD uses the SQuAD question-answering format [2], and our clause-text model uses the standard SQuAD start/end-token setup, so the data works with common question-answering tools.
- **Licences and credit.** The dataset, the pre-trained models and all software are used under their open-source licences (CUAD under CC BY 4.0; DistilBERT, FLAN-T5, `transformers` and Streamlit under Apache 2.0; PyTorch and scikit-learn under BSD licences), and they are credited in the references.
- **Testing practice.** We followed standard machine-learning practice: a separate test set split by contract, no tuning on the test set, confidence intervals from a bootstrap over contracts [18], and standard scores (F1, exact match, ROUGE-L [8]).
- **Repeatability.** Appendix B gives a checklist covering the software versions, data, seeds, file checksums and commands, and all code and reports are kept in Git.
- **Writing.** References use the IEEE style, and this report follows the Brac University CSE400 report template.

3.6 Project Management Plan

3.6.1 Schedule

The project was done by a team of three during 2026. Table 3.2 shows the main stages and what each one produced.

3.6.2 Budget

The project cost no money. The dataset, the pre-trained models and all software are free, and all training and testing ran on a laptop the team already owned. No cloud computing, paid services or paid labelling were used.

3.6.3 Resource management

The one shared laptop was the scarcest resource. Training runs were done one at a time, because the MPS backend does not free GPU memory between runs in the same program; the summarizer was trained on the CPU for the same reason (Section 4.4.6). Everything needed for testing (the split, the baseline model and the three trained models) was saved to disk, so tests could be repeated without retraining. Code, notebooks and reports were kept in Git.

Table 3.2: Project schedule

Period (2026)	Activities	Deliverables
Before June	Problem definition, literature review, initial design including a cross-clause rule engine	Project proposal
June	Data acquisition, wide-to-long preprocessing, exploratory data analysis	EDA notebook
July	Contract-level 80/20 split; TF-IDF baseline; training of the presence, span and summarizer models; standalone evaluation; Clausify application	Trained models, <code>test.ipynb</code> , Clausify prototype
August	Supervisor review: training-budget and seed controls, cluster bootstrap, span and end-to-end evaluation; second review: risk-weighted evaluation, threshold and calibration, aggregation ablation, split variance	Project and findings reports, analysis scripts
September	Write-up of the full report and a conference-style paper; presentation	Report, paper, presentation slides
October	Viva preparation; re-training and re-testing of each model; final report in the CSE400 template	Final project report

3.7 Risk Management

Table 3.3 lists the main risks we found during the project and what we did about each one. Several of them actually happened, and the last column shows what was done.

3.8 Economic Analysis

Development cost. The project cost no money: the data, models and software are free, and the laptop was already owned (Section 3.6). The real cost was the team’s time and about ten hours of laptop computing.

Running cost. The models run on a CPU with about 1.5 GB of memory, so the system can run on ordinary hardware with no cost per contract. Table 3.4 compares this with the two options a user has today.

Benefits. The main benefit is quick sorting: pointing the reader to the parts of

Table 3.3: Risk management plan

Risk	Impact	Mitigation
First-choice backbone unstable on available hardware	High	Occurred (DeBERTa-v3 produced NaN gradients on MPS). Switched to DistilBERT; the method is unchanged and the backbone can be swapped back on CUDA hardware.
GPU memory exhaustion during training	Medium	Occurred. One training run per process; summarizer trained on CPU.
Data leakage between train and test	High	Contract-level split reused by all three models; retrieval and inference never use gold offsets; ensemble is parameter-free (Section 4.2.5).
Optimistic results from test-set tuning	High	No hyperparameter was tuned; all settings are published defaults or fixed by arguments made before training (Section 4.4.1).
Over-claiming small differences between models	Medium	Cluster-bootstrap confidence intervals, seed, budget and split controls (Section 5.4.1 onwards).
An architecture that cannot reach the required recall	High	Retrieval ceiling measured before training (64%), which ruled out the retrieval design at no training cost (Section 4.2.1).
Schedule overrun from long training runs	Medium	Distilled models (about 1.5 hours total training); subsampled training sets; saved artifacts so evaluation needs no retraining.
Loss of work or results	Medium	Version control on GitHub; trained weights backed up before any re-training.
Users over-trusting the application	High	Not-legal-advice notice; risk badge presented as more reliable than quoted text; limitations stated in the application (Section 5.6.2).

a long contract that most likely matter and explaining them in plain language, at almost no cost. Expert review in this field is very valuable (the CUAD labels alone are said to represent about two million US dollars of lawyers’ time [1]), so even saving part of the time needed to find key clauses is worth something.

Limits of the benefit. The value depends on how reliable the system is. Only 22.0% of real clauses get through the whole system, and the main setup misses 36.9% of High-risk clauses (Section 5.2.2). So the system cannot replace a lawyer. Its value is as a free first look, not as a replacement for professional review.

Table 3.4: Cost–benefit comparison

	Lawyer review	Hosted LLM service	This system (local)
Marginal cost per contract	High (professional fees)	Per-request fee	Near zero
Time per contract	Hours to days	Seconds	About 30 seconds
Privacy	Confidential	Contract leaves the user’s machine	Contract stays on the machine
Reliability	Expert judgement	Unmeasured on this task	Measured; misses many clauses
Legal standing	Legal advice	None	None

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

The design followed the objectives and constraints of Chapter 3 through an evaluate-first process with four rules: (i) build a strong, simple baseline before any complex model; (ii) measure the feasibility ceiling of a design before spending training time on it; (iii) fix the contract-level train/test split once and reuse it everywhere, never tuning on the test set; and (iv) evaluate each stage on its own and then the whole pipeline end to end. The resulting system is a three-stage pipeline with a risk layer on top.

The system is a staged pipeline (Figure 4.1). Contract text is windowed; a presence stage decides which of the 41 categories the contract contains; a span stage extracts the exact clause text for each detected category; a risk layer attaches the detected *category's* High/Medium/Low level; and a text-generation stage attaches a plain-English sentence. The final output is a risk-prioritized report. (The deployed Clausify application runs every stage; Section 4.4.9.) Section 5.1.3 shows that the last stage does not in fact summarize the clause in front of it, it selects among 41 pre-written category sentences, so the pipeline is better described as classification-to-template than as summarization. Figure 4.1 names the stages and the models behind them; Figure 4.12 at the end of this chapter repeats the same chain with the concrete data object that crosses each boundary, which is the version to consult when reading the implementation.

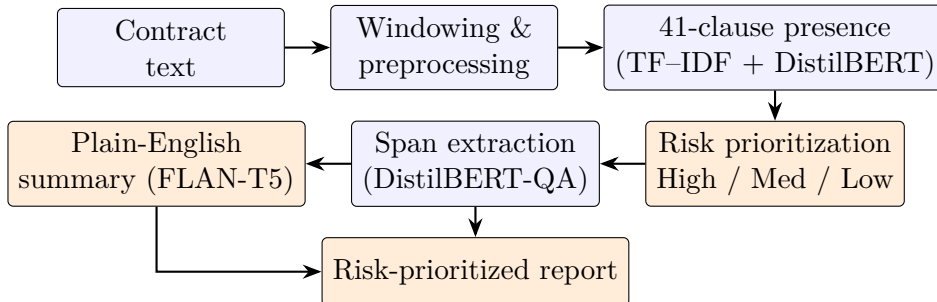


Figure 4.1: System pipeline

One principle runs through everything in this chapter: **build a strong, cheap**

baseline first, and measure feasibility ceilings before spending training time. Both habits changed the design in concrete ways, as the next sections show.

4.2 Preliminary Design or Design (Model) Specification

For each stage, alternative designs were considered and, where possible, checked by measurement before one was selected. For presence classification three alternatives were compared (a full-document lexical baseline, a retrieval-then-classify transformer, and a sliding-window multiple-instance transformer) together with three ways of combining the baseline and the transformer. Sparse-attention architectures were considered and rejected on the grounds given in Section 2.1.2, and their cost was later benchmarked (Section 4.4.7). The specifications of the selected design for each stage follow.

4.2.1 Stage 2A: Presence Classification

A strong baseline first: full-document TF-IDF

Classical bag-of-words models have no input-length limit, so a TF-IDF representation gets to read the *entire* contract: an advantage no windowed transformer has. We fit a TF-IDF vectorizer (20,000 features, word uni- and bi-grams, sublinear TF, $\text{min_df} = 2$) on the 408 training contracts and train one class-balanced logistic regression per category. Legal drafting is extremely formulaic (“*This Agreement shall be governed by the laws of the State of ...*”), so distinctive vocabulary alone carries a lot of signal. This baseline sets the bar every neural model has to beat.

Measuring the retrieval ceiling, and dropping retrieval

The obvious way to hand a long contract to a transformer is retrieval: slide 2,000-character windows with a 1,500-character stride over the text (the windowing scheme itself is drawn in Figure 4.3), score each window by TF-IDF cosine similarity to the category query, and give the model only the best window or windows. The query uses only information available at inference time (the category name or its CUAD question, never the answer) so there is no leakage.

Before training anything, we measured the *retrieval hit-rate*: how often does the selected window actually contain the gold clause? Figure 4.2 shows what the measurement is doing and why the answer is a hard bound. Whatever this number is, it is an upper bound on the downstream model’s recall, because a clause sitting in a window the retriever never selects cannot be found by any model placed after it. The best strategy we found (category name as query, top-3 windows) reached only **64.0%** (Table 4.1). Interestingly, the CUAD question text did worse than the bare category name, because the questions share so much boilerplate that all the category queries end up nearly identical. A recall ceiling of 0.64 sits well below the baseline’s recall of 0.81 (and its micro-F1 of 0.775), so a retrieval-windowed transformer was

going to lose before it was ever trained. The measurement took seconds and saved us a wasted training run.

One qualification belongs with that number. What we measured is a ceiling on *lexical* retrieval: every query strategy in Table 4.1 scores windows by TF-IDF cosine similarity, so a clause phrased without the vocabulary of its category name is invisible to the retriever even when it is plainly present. A dense retriever, a sentence encoder embedding both the query and each window, matches on meaning rather than on shared terms and would very likely lift the 64.0% figure, possibly above the baseline. We did not test one, so the honest reading of this section is narrower than “retrieval does not work”: it is that *lexical* retrieval imposes a ceiling too low to build on, which was enough to send us to the windowing design in Section 4.2.1. Whether a dense retriever raises the ceiling far enough to make a retrieve-then-read pipeline competitive is an open question we list in future work.

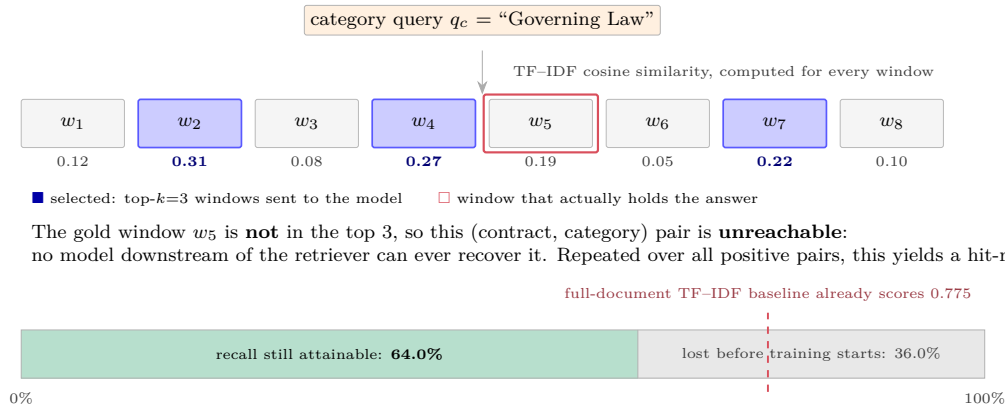


Figure 4.2: Retrieval ceiling

Table 4.1: Retrieval hit-rates

Query strategy	Top- k	Hit-rate
CUAD question text	1	38.6%
Category name	1	44.6%
Category name	3	64.0%
Name + question	3	61.5%

Note. The hit-rate is the recall ceiling a retrieval-based design would impose.

Sliding-window max-pooling (multiple-instance learning)

To get rid of the ceiling we trained a *window-level* classifier instead: given the pair (category question, window), predict whether that window contains a clause of the category. Figure 4.3 shows how the windows are generated. A 2,000-character window advancing in 1,500-character steps leaves a 500-character overlap between neighbours, and that overlap is what guarantees no clause can fall between two windows and disappear: the median annotated clause is 196 characters (Section 4.3.5), comfortably shorter than the overlap, so every clause lands wholly inside at least one window.

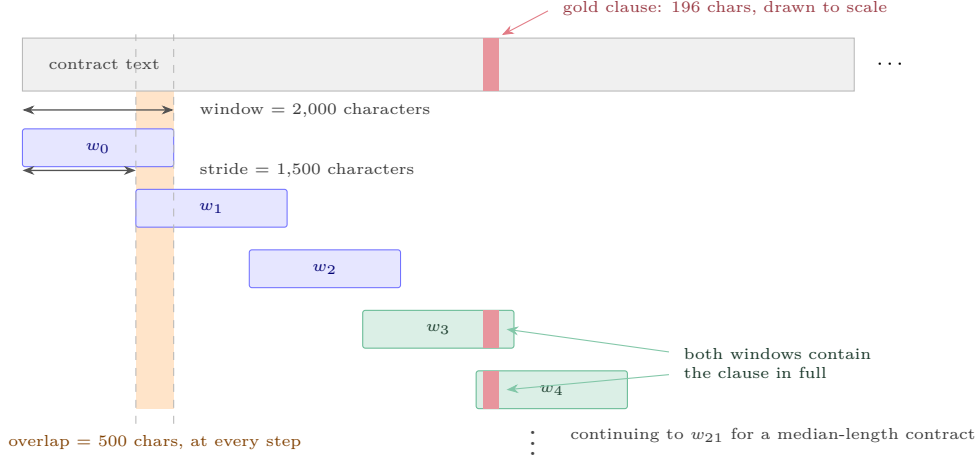


Figure 4.3: Sliding-window generation

Training labels come from the gold character offsets: any window containing an annotated span is positive, and for each (contract, category) bag we sample two negative windows. Using gold offsets this way is ordinary supervision, not leakage: the labels shape the training targets, but they never touch the model’s input, and nothing gold is used at inference. From the 408 training contracts this gives 53,662 window examples (39.2% positive).

At inference the model scores *every* window of the contract, and the document-level prediction is simply the maximum window probability: the multiple-instance learning decision rule, drawn in Figure 4.4. The document is a *bag* of window instances; the bag is positive if any instance is positive; and max implements exactly that rule. Since the windows tile the whole document with overlap, every present clause is reachable, so the recall ceiling disappears by construction. The price is precision, and Figure 4.4 also shows why: the maximum is taken over every window, so a single spurious window anywhere in the document is enough to flip the whole contract to positive. With a median of 22 windows per contract and a mean of 35, the chance that some window crosses the 0.5 threshold by accident grows with document length: long contracts are structurally more likely to produce a false positive than short ones, independently of what they actually contain.

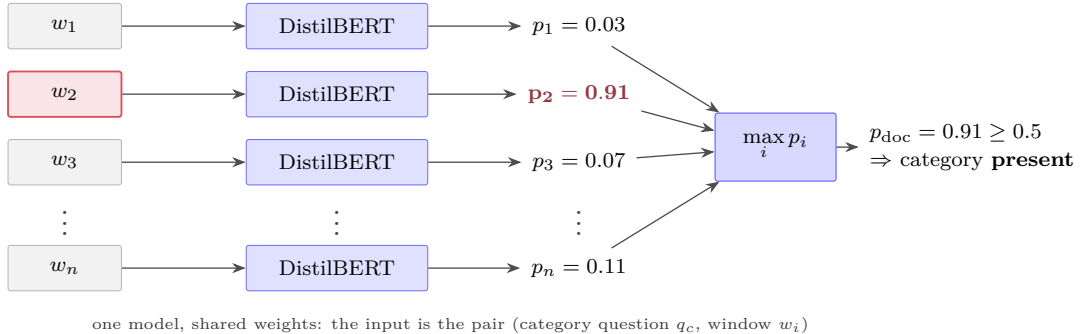
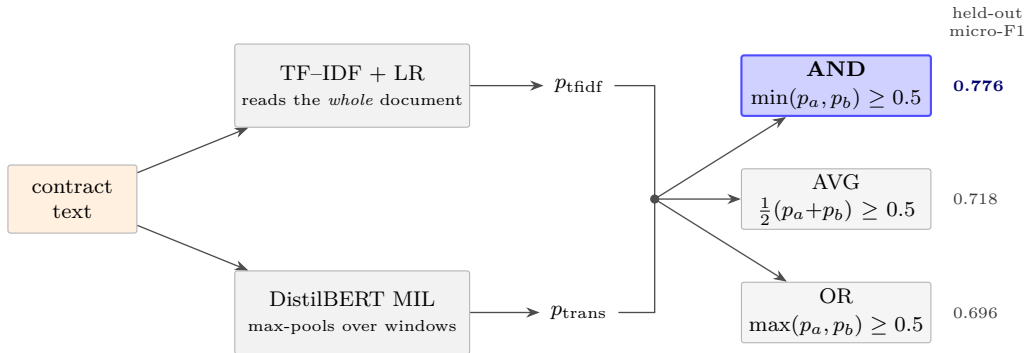


Figure 4.4: Multiple-instance max-pooling

The conjunctive ensemble

When we compared the two models category by category, they turned out to be *complementary*: the transformer wins on semantically subtle categories (for example *No-Solicit of Employees* and *Liquidated Damages*), while the baseline wins on keyword-heavy ones. We combine them with **parameter-free** rules (AND, the min of the two probabilities; OR, the max; and AVG, the mean), chosen deliberately because they contain nothing tunable, so nothing can overfit to the test set. Figure 4.5 lays out the decision. The AND rule, which reports a clause only when both models agree, filters out each model’s uncorrelated false positives: for a spurious detection to survive, the lexical model and the windowed transformer must both make the same mistake on the same contract and category, and their mistakes are largely independent. That recovers exactly the precision that max-pooling loses.



All three rules are **parameter-free**: there is nothing to fit, so the test set cannot tune them. AND fires only when *both* models cross the threshold, so a false positive has to be made twice, independently, to survive.

Figure 4.5: Ensemble rules

4.2.2 Stage 2B: Span Extraction

Here the transformer has no substitute: a bag-of-words model cannot point at character positions. We fine-tune `DistilBertForQuestionAnswering`: the input is (CUAD question, window text) and the model predicts start and end token positions. Character offsets from `CUAD_v1.json` are converted to token labels through the tokenizer’s offset mapping.

Two different window sizes are in play in this project, and it is worth being explicit about the difference, since mixing them up makes the whole design look inconsistent. The 2,000-character window of Section 4.2.1 is a *presence* window: it tiles the document blindly, because at that point we do not know where anything is. The window used here is a *focus* window: 1,200 characters, and it is not tiled at all but re-centred on the gold answer, positioned so the answer begins roughly 150 characters in. Figure 4.6 draws the two on the same scale. The reason for the smaller, re-centred window is purely mechanical: it guarantees the target span survives truncation at the model’s 320-token limit, which a blind 2,000-character window does not: an answer sitting at the tail of a presence window would simply be cut off, and the model would be trained on an example whose answer is not present. Note also

that this stage has no length ceiling at all: extraction happens *inside* a window that Stage 2A has already flagged as relevant.

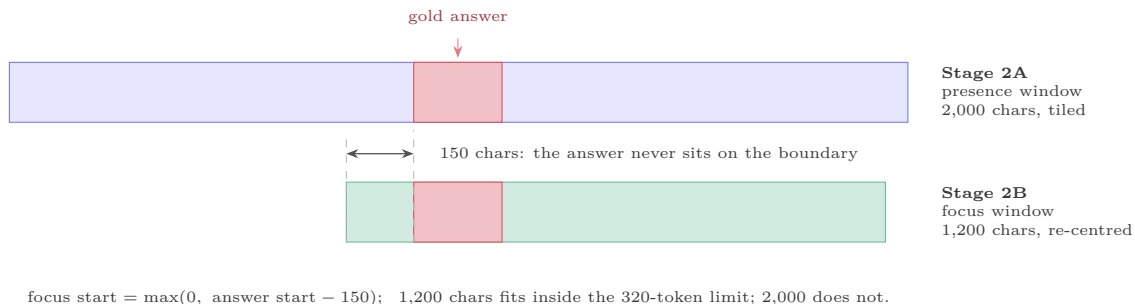


Figure 4.6: Presence vs. focus window

4.2.3 Stage 1: Plain-Language Summarization

CUAD gives us clauses but no plain-English rewrites to learn from, so we wrote our own seed: **41 gold templates**, one carefully phrased single-sentence summary per category, framed from the weaker party’s perspective. For *Anti-Assignment*, for instance, the template reads “You cannot transfer this contract to someone else without permission.” Every extracted training clause is paired with its category’s template, which yields 11,156 training pairs, and FLAN-T5-small is fine-tuned on the prompt “summarize in plain English: <clause>”.

We evaluate with ROUGE-L *and* with a side-by-side zero-shot-versus-fine-tuned comparison on the same clauses. The double evaluation is deliberate. With only 41 unique target sentences, a model can earn a high ROUGE score just by learning to classify the clause and emit its template word for word, without doing any real summarization. Chapter 5 shows that this is exactly what happens, and we report it as such.

4.2.4 Category-Level Risk Prioritization

A naming point first, because it governs what the rest of this section may claim. The system does not read a clause and judge how dangerous *that* clause is. It identifies which of the 41 categories a contract contains and attaches a fixed level to the category. Those are different tasks, and conflating them would overstate the system considerably: our layer says “*contracts containing a non-compete deserve attention*”, never “*this non-compete is unusually harsh*”. We therefore call it **category-level risk prioritization** rather than risk classification throughout.

Assignment criteria

The levels were not assigned by intuition alone, and the criteria we used should be stated so that a reader, or a reviewing lawyer, can disagree with a specific assignment on specific grounds. Each category was scored on five dimensions, from the weaker party’s side:

1. **Financial exposure.** Can this clause cost money, and is the amount bounded? Unbounded exposure is the strongest single indicator of High.
2. **Irreversibility.** If the term operates, can the position be recovered? Losing ownership of intellectual property or granting a perpetual licence cannot be undone by later negotiation; a notice period can.
3. **Restriction of future freedom.** Does the clause constrain what the party may do *after* the contract, or outside it: whom it may work for, sell to, or hire?
4. **Asymmetry.** Does the obligation fall on one side only? A mutual confidentiality duty is less dangerous than a one-way indemnity of identical wording.
5. **Duration and survival.** Does the obligation outlive the agreement?

The rule we applied: **High** if the category is unbounded on dimension 1, or irreversible on dimension 2, or restricts post-term freedom on dimension 3. **Low** if the category is informational (it records a fact such as a date, a name or a governing jurisdiction, or confers a benefit) and scores on none of the five. **Medium** otherwise: a real obligation, bounded and reversible. This is why *Cap on Liability* is Medium and not Low despite sounding protective (it bounds recovery, which can under-compensate a genuine loss), and why *Governing Law* is Low despite being consequential in litigation (it is a fact about the contract, not an obligation falling asymmetrically on the signer).

The eight High categories fall into three groups, each triggered by a different dimension: money you could owe without limit (*Uncapped Liability*, *Liquidated Damages*, *Minimum Commitment*, *Most Favored Nation*); something you cannot get back (*IP Ownership Assignment*, *Irrevocable/Perpetual License*); and freedom to earn later (*Non-Compete*, *Exclusivity*). The full clause-by-clause table with reasons is in Appendix A, and Figure 4.7 shows the distribution.

Who is the “weaker party”?

This phrase does real work in the taxonomy and it is ambiguous, so we should say how we resolved it. The weaker party is not a fixed role (in a distribution agreement it may be the distributor, in a licence the licensee, in a franchise the franchisee) and in a genuinely balanced agreement between equals there may be no weaker party at all.

We operationalized it as follows: **the party with less bargaining power over the drafting**, which in the CUAD corpus is in practice the party that did not write the agreement. Where that is genuinely unclear from the category alone, we scored from the position of **the party on whom the obligation falls**, so *Non-Compete* is scored from the side restrained, *Audit Rights* from the side audited, *License Grant* from the side receiving. This is a heuristic, and it has a known failure mode worth stating: a clause that is High-risk for one side is often protective for the other, so the same badge shown to a licensor and a licensee is not equally informative. A per-instance system would resolve the perspective from the clause text; ours cannot,

and assumes the user is the party with less power. That assumption is stated in the notice the application shows above every analysis.

Why a lookup table rather than a learned score

The taxonomy is a transparent lookup rather than a model output, and that is deliberate: it can be audited line by line, its reasoning is written down per category, and it cannot drift silently between versions. The cost of that choice is precisely the limitation named at the top of this section: it cannot distinguish a \$10,000 liability cap from a \$10,000,000 one. Section 5.2.2 measures a second cost we did not anticipate: because the levels are fixed per category, the system’s error profile is set entirely by which *categories* it detects well, and the categories carrying High risk turn out to be among the ones it detects worst.

Provenance and validation status of the taxonomy

This taxonomy is the main thing this project adds on top of CUAD, and it is also the component with the weakest evidential backing, so its provenance should be stated exactly rather than left to be inferred.

Who wrote it. All 41 assignments were made by the three of us. We are computer science students, not lawyers. The levels were assigned by reasoning about consequences for the weaker party (can this clause cost you unbounded money, take away something you cannot get back, or restrict what you may do afterwards) and cross-checked against the CUAD category descriptions, but they rest on our judgement alone. No legal professional has reviewed them.

Why that matters more here than elsewhere. Every other component in this project is measured against gold annotations that experts produced. This one is not measured against anything. It also has the most direct route to the user: the level decides what Clausify tells a signer to worry about first, so an error here is not a lower score in a table, it is a warning that does not appear. A category we have marked Medium that a practitioner would mark High is a silent failure of the deployed system.

The check that is needed, and its status. What would establish face validity is a review by someone with legal training: hand over the 41 categories with our level and our stated reason, ask for agreement or disagreement on each, and report the disagreement count *whatever it turns out to be*. We have prepared that review instrument (it is in the repository as `review/risk_taxonomy_review_sheet.csv`, one row per category with columns for the reviewer’s own level and comment) and it takes a reviewer roughly half an hour.

A single reviewer would establish face validity; *two or three, reviewing independently*, would allow something stronger, and the difference is worth explaining. Raw percentage agreement is a poor statistic on this taxonomy because the levels are unevenly distributed: 22 of the 41 categories are Medium, so a reviewer who answered “Medium” throughout would score 54% agreement while contributing no information at all. Chance-corrected agreement (Cohen’s κ for a pair, Fleiss’ κ across three or more raters) removes exactly that artefact, and is what we would report. The

repository includes `review/agreement.py`, which takes the completed sheets and prints per-reviewer agreement, both kappas, and the list of disputed categories, so that the analysis is fixed in advance and cannot be shaped by the result.

At the time of writing this review has not been carried out, and no number from it is reported anywhere in this project. Until it is, the taxonomy should be read as a documented, auditable, and explicitly unvalidated design artefact: its virtue is that every assignment is written down with a reason and can be argued with line by line, not that anyone qualified has yet agreed with it.

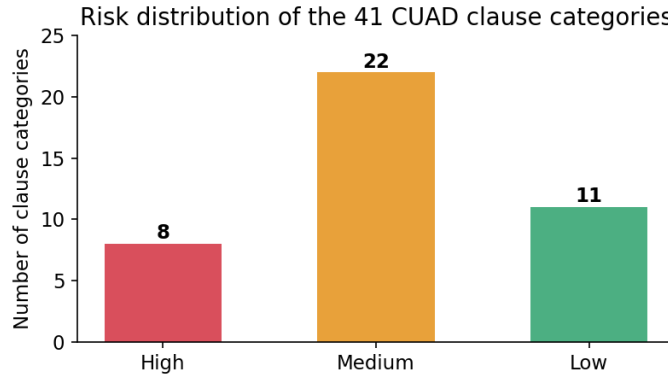


Figure 4.7: Risk-level distribution

4.2.5 Leakage Discipline

With a dataset this small, leakage is the most dangerous way to fool ourselves, so we kept one explicit rule the whole way through: *labels may shape training targets, but they may never shape the input a model sees or any inference-time selection.* In practice this meant: the split is contract-level; retrieval queries use only the category or question; gold offsets label training windows but never build inputs or pick inference windows; and the ensemble rules are parameter-free, so the test set cannot tune them.

One consequence is visible in Figure 4.6 and deserves stating outright, because it is the one place where the rule looks like it might be broken. The focus window is positioned using the gold answer offset, which is a label. In training that is ordinary supervision: it decides which text becomes a training example. In the deployed pipeline there is no gold offset, and Stage 2B runs over the presence window that Stage 2A selected. The *isolated* span benchmark of Section 5.1.2 is the exception, and we state it plainly: it builds each test example with the same re-centred focus window, so it measures extraction *when the answer’s location is already known*. That makes it an oracle upper bound, not a deployable score; the realistic number is the end-to-end measurement of Section 5.2.1, which never uses a gold offset.

4.3 Data Collection

All experiments use the official Atticus Project release of CUAD, obtained from the Hugging Face hub (`theatticusproject/cuad`). One practical discovery is worth recording here: the repository’s default `load_dataset()` configuration exposes only the 511 raw contract PDFs, which cannot be trained on. The annotated data actually lives in two files inside the repository, and each one plays a different role in our pipeline:

- `master_clauses.csv`: an 83-column wide table with one row per contract, which we used for exploratory analysis and for building labels;
- `CUAD_v1.json`: the SQuAD-format file with the full contract text and character-level answer offsets, which we used for span-extraction training and full-document context.

4.3.1 Data Cleaning

The CUAD files are expert-curated, so cleaning concerned format inconsistencies rather than noisy values. Four issues had to be handled:

- **Missing cells.** Clause-text cells that are empty (NaN) are treated as an empty clause list, i.e. the category is absent.
- **Malformed list literals.** Each clause-text cell holds a stringified Python list. Cells that do not parse as a list literal are kept as a single clause string rather than discarded, and whitespace-only entries are removed from every list.
- **Inconsistent column headers.** The header `Notice Period To Terminate Renewal- Answer` contains a stray space that breaks name-based matching; columns were therefore paired by position (Section 4.3).
- **Empty answers in the JSON file.** Answer spans whose text is empty after stripping whitespace are ignored when building span-extraction examples and gold-span lists.

No outlier removal was applied: very long contracts are genuine and were kept, because the length problem is precisely what the method must handle.

4.3.2 Data Transformation

The CSV is wide: 41 categories $\times$ (a clause-text column plus an answer column), with a filename column in front. Turning it into a long table of (contract, category) rows forced two decisions that were not obvious at first:

Pair columns by position, not by name. The header `Notice Period To Terminate Renewal- Answer` contains an inconsistent space, which silently breaks any string matching between a category column and its answer column. Pairing by column position (clause text at odd indices, answer at even) survives typos like this.

Define presence from the clause-text list, not the Yes/No answer. Each clause-text cell holds a stringified Python list, either '[]' or "['clause', ...]". We call a category *present* when this list is non-empty. This rule works the same way across the eight entity-style categories (dates, party names) whose answer column holds a value instead of Yes/No, and it hands us the clause spans as a side effect.

The melt produces $510 \times 41 = 20,910$ labelled (contract, category) rows, of which 32.1% are positive.

Three further transformations prepare the data for the models. **Windowing:** every contract is cut into 2,000-character windows with a 1,500-character stride, so consecutive windows overlap by 500 characters, more than the median clause length, and no clause falls between two windows. **Offset conversion:** for span extraction, CUAD’s character offsets are converted to start and end token positions using the tokenizer’s offset mapping, inside a 1,200-character focus window around each answer so that it fits in 320 tokens. **Text vectorization:** for the baseline, each full contract is represented as a TF-IDF vector of unigrams and bigrams with sublinear term frequency. Category labels are kept as binary present/absent indicators; no scaling or normalization of numeric features is required beyond what TF-IDF performs.

4.3.3 Data Integration

The two CUAD files are complementary. `master_clauses.csv` provides one row per contract with the clause lists used for exploratory analysis and labels, while `CUAD_v1.json` provides the full contract text, the 41 category questions and the character offsets of every annotated span. The JSON file is the primary source for all modelling: presence labels, gold spans and contexts are all derived from it, keyed by contract title and by the category name parsed from each question identifier. The CSV is used for the exploratory analysis and to cross-check the per-category positive rates. The seeded contract-level split is computed once over the 510 contract titles and applied identically to every derived table.

4.3.4 Data Reduction

Reduction was used to keep training feasible on a laptop and to control class balance, not to discard information at inference time:

- **Feature selection for the baseline.** The TF-IDF vocabulary is limited to the 20,000 most frequent unigrams and bigrams that occur in at least two contracts.
- **Negative sampling.** For window-level presence training, every positive window is kept but only two negative windows are sampled per (contract, category) bag, giving 53,662 window examples with a 39.2% positive rate.
- **Subsampling.** The presence model was trained on a stratified subsample of 24,000 windows, the span extractor on 8,000 of 11,156 examples and the

summarizer on 4,000 of 11,156 pairs. Section 5.4.3 tests whether the presence subsample handicapped the transformer.

4.3.5 Summary of Preprocessed Data

Four properties of the data ended up shaping every design decision that followed:

1. **Severe class imbalance.** Per-category positive rates run from *Document Name* at 100% and *Parties* at 99.8% down to *Source Code Escrow* at 2.5% and *Price Restrictions* at 2.9%. The median category is present in roughly 23% of contracts.
2. **The rare tail is nearly few-shot.** *Source Code Escrow* has only 13 positive contracts in the entire corpus. No model learns categories like that reliably, and per-category scores on this tail are mostly noise. We carry this caveat through the results chapter.
3. **Contracts are long.** Median 33,143 characters (about 5,000 words), maximum 338,211 characters. The median is roughly $16\times$ a transformer’s effective input window of about 2,000 characters, and the longest about $170\times$. This is the central modelling tension of the whole project.
4. **Clauses are short.** The median annotated clause is 196 characters (90th percentile 587, 99th percentile 1,330). So a clause fits easily inside a single window; the hard part is finding it inside a very long document.

It is worth being upfront that 510 contracts is the entire dataset. Its annotations reportedly cost about \$2M of attorney time, so it cannot simply be grown. Three consequences follow: transfer learning from pre-trained models is mandatory; overfitting is the main enemy, so all our conclusions rest on held-out numbers; and the train/test boundary has to be drawn at the contract level.

The 80/20 Train/Test Split

Following the evaluation protocol set by our supervisor, we split the dataset **80/20 at the contract level**: whole contracts go to one side or the other, so no clause from a training contract can leak into the test set. The split is generated with a fixed seed (42) and *reused identically by all three models*, which means “test” always refers to the same 102 unseen contracts. Table 4.2 summarizes the split; the nearly identical positive rates on both sides confirm it is balanced. We exported the two sides as `data/train_80.csv` and `data/test_20.csv` so the split itself is reproducible.

The aggregate positive rates in Table 4.2 are almost identical (32.0% vs. 32.2%), but a single pair of percentages can hide a badly skewed split: two sides can agree overall while disagreeing sharply category by category. Figure 4.8 therefore compares the two sides for all 41 categories at once. The profiles track each other closely across the whole frequency range (Pearson $r = 0.986$, mean absolute difference 3.4 percentage points). The visible exception is *Termination For Convenience*, which appears in 47.1% of test contracts against 33.1% of training contracts, a 14.0-point gap; the

Table 4.2: Train/test split

	Contracts	(Contract, category) rows	Positive rate
Train (80%)	408	16,728	32.0%
Test (20%)	102	4,182	32.2%
Total	510	20,910	32.1%

other outliers are all in the rare tail, where a shift of one or two contracts moves the percentage several points on its own. Since the split was drawn once at random and never re-drawn to look better, we report these gaps rather than tune them away, but they are worth carrying into Chapter 5, because a category over-represented in the test set carries more weight in the micro-averaged score than its training frequency would suggest.

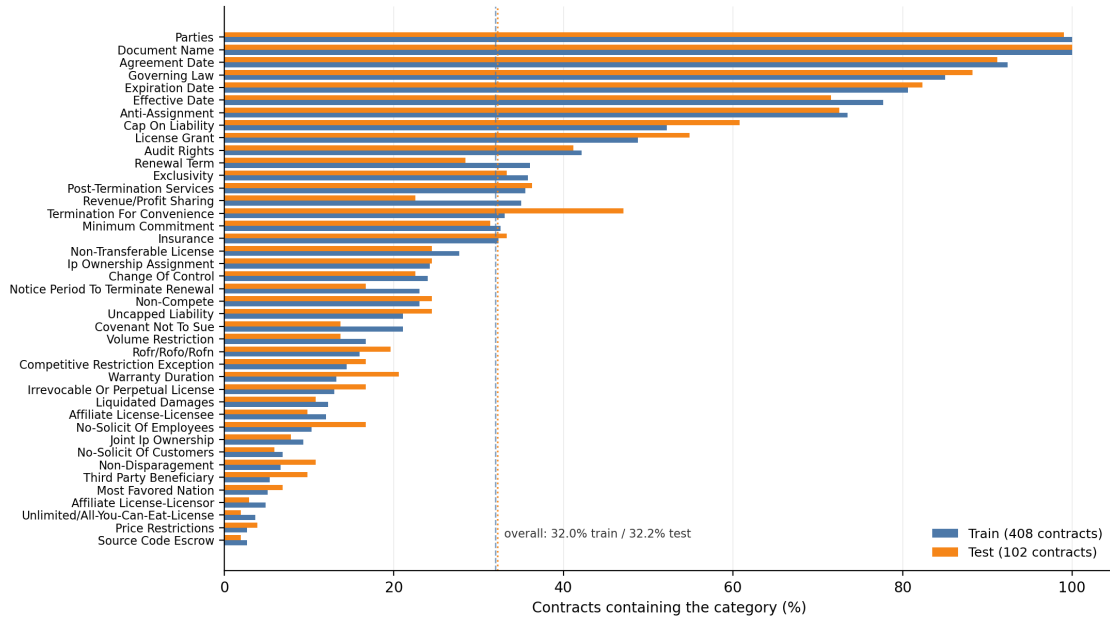


Figure 4.8: Per-category split balance

4.4 Implementation of Selected Design

The selected design was implemented in Python as two Jupyter notebooks, `train.ipynb` for data preparation and training and `test.ipynb` for standalone evaluation, supported by analysis scripts, and deployed as the Clausify web application. This section describes the training of each model and the application.

All models were trained on the 408 training contracts (80%) only. Training ran on a single Apple-Silicon laptop using the MPS (Metal) backend with `torch` 2.8 and `transformers` 5.x; the summarizer ran on CPU for the memory reasons explained in Section 4.4.6. Table 4.3 collects the configurations.

Table 4.3: Training configurations

	Presence (MIL)	Span (QA)	Summarizer
Backbone	DistilBERT-base	DistilBERT-base	FLAN-T5-small
Task head	Sequence classification	QA (start/end)	Seq2seq LM
Training examples	24,000 (of 53,662)	8,000 (of 11,156)	4,000 (of 11,156)
Max input length	256 tokens	320 tokens	256 in / 48 out
Epochs	2	2	2
Batch size	16	16	8
Learning rate	2×10^{-5}	3×10^{-5}	3×10^{-4}
Device	MPS	MPS	CPU
Wall-clock	$\approx 50\text{--}52$ min	$\approx 23\text{--}26$ min	$\approx 8\text{--}13$ min

4.4.1 Validation Protocol, and Where the Hyperparameters Came From

An examiner is entitled to ask how the settings in Table 4.3 were chosen, because if any of them were selected by trying variants and keeping whichever scored best on the 20% test set, then every number in Chapter 5 is optimistic and the held-out evaluation is not held out. This section answers that directly, including one place where our practice was worse than it should have been.

What was tuned, and what was not

Nothing in Table 4.3 was tuned. Every hyperparameter is either a published default for the backbone or a value fixed by an argument made before training, and no variant of any of them was ever compared on the test set. Concretely:

- **Learning rates, batch sizes, epochs** (2×10^{-5} , 3×10^{-5} , 3×10^{-4} ; 16/16/8; 2 epochs) are the standard fine-tuning defaults for DistilBERT and T5-family models. We ran each configuration once. No sweep was performed, which protects the test set, but also means we have no evidence that these are good values, only that they are conventional ones.
- **Window 2,000 characters, stride 1,500** follows from a measurement on the *training* data, not from tuning: the 500-character overlap has to exceed the median annotated clause (196 characters) so that no clause falls between windows (Section 4.2.1). Any window/stride pair satisfying that would do; we did not search for the best one.
- **Two negative windows per bag** is a class-balance decision, chosen to hold the positive rate near 39% rather than let the vast majority of windows be negative. Not tuned.
- **Max-pooling** was chosen because it *is* the multiple-instance decision rule, not because it beat alternatives: we never tried alternatives. Section 5.3.1 corrects that omission after the fact.

- **Threshold 0.5** is the default decision boundary of a two-class softmax. It was never chosen, examined, or adjusted. Section 5.3.2 shows, after the fact, that it is a poor operating point for the max-pooled transformer.

The honest summary is that this project avoided test-set tuning by not tuning at all. That is a real guarantee about the validity of the held-out numbers, and simultaneously a limitation: the transformer may be losing to the baseline partly because nobody ever looked for better settings for it. We record it in both directions.

The validation sets actually used

Table 4.4 states, per model, what was held out during training and what it was used for.

Table 4.4: Validation arrangements

Model	Validation data	Use
Presence (MIL)	5% of the <i>training</i> windows, split off before training	Monitoring only. Clean: no test contract contributes.
Span (QA)	None (<code>eval_strategy="no"</code>)	No validation loss exists for this model.
Summarizer (deployed checkpoint)	None	The checkpoint used in every reported result and in Clausify was trained with no evaluation set.

Note. “Monitoring only” means the loss was printed and plotted but never used to select a checkpoint, stop early, or change a setting: all three runs used `save_strategy="no"` and kept the final-step weights.

One thing we got wrong

The exploratory notebook run of the summarizer, the run whose loss curve appeared in earlier drafts of this report, used 300 clauses *drawn from the test split* as its evaluation set. That is the wrong data to monitor on, and we should not have done it.

Two things limit the damage, and we state them without treating either as an excuse. First, the run was configured with `save_strategy="no"` and no early stopping, so the numbers it printed could not select a checkpoint; nothing about the model depended on them. Second, the deployed summarizer was rebuilt by `build_artifacts.py` with *no* evaluation set at all, and it is that checkpoint, not the notebook’s, that produced every summarization result in Chapter 5. So no reported score was computed by a model that had been selected using test data.

What remained was that the validation loss printed by that run ($0.096 \rightarrow 0.075$) was a loss on held-out *test* clauses mislabelled as validation, and should be discounted. The October 2026 re-training rebuilt the summarizer exactly as it is deployed, with

no evaluation set, and Figure 4.11 now plots that run’s training loss, so no test data appears in it. We record the mistake rather than drop it, because it is more instructive than a clean curve would be: with a small dataset and three models it is easy to reach for “the other split” when a validation set is needed, and the discipline that catches it is writing down, per model, exactly what was held out, which is what Table 4.4 now does.

4.4.2 Optimization Schedule

Table 4.3 gives the peak learning rates, but a peak rate on its own does not describe how a run behaved: the schedule that decays it does. All three models use the same optimizer configuration: AdamW with a **linear decay** schedule, **no warmup**, and gradient-norm clipping at 1.0. Figure 4.9 plots the three resulting schedules. The presence and span settings were read back from the `training_args.bin` saved beside each checkpoint rather than from our notes, so what is plotted is what actually ran.

Two choices are worth defending. Skipping warmup is normally risky, but these are short fine-tuning runs (1,000–3,000 steps) starting from pre-trained weights at small learning rates, where the usual justification for warmup, stabilizing early updates in a randomly initialized model, barely applies; the smooth loss curves in Figures 4.10 and 4.11 show no early instability. Clipping at a gradient norm of 1.0 was left on throughout, which matters more than it might seem on the MPS backend: it is the guard that keeps a single bad batch from producing the kind of divergence we hit with DeBERTa-v3 (Section 4.4.6).

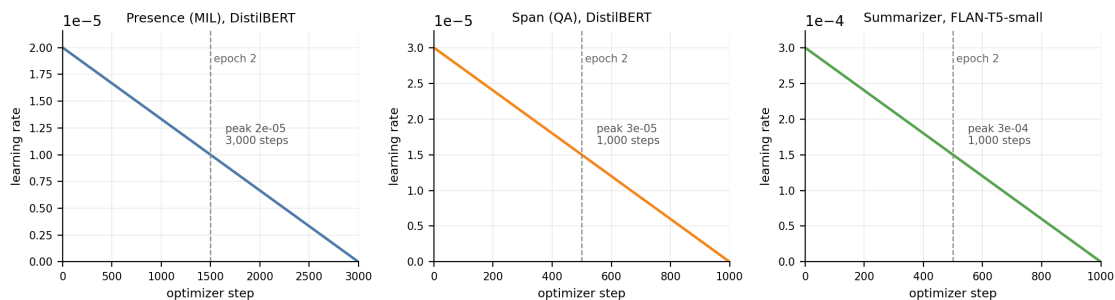


Figure 4.9: Learning-rate schedules

We should be straightforward about one limitation, and about how far we were able to repair it. A per-step gradient-norm trace is the natural companion to these curves, and it is the one diagnostic that lets a reader confirm the runs stayed numerically healthy instead of inferring it from the loss. We did not retain it for the three original runs: the intermediate checkpoint directories were cleared after training, so only the final weights and the loss values quoted below survive from them.

We did retain it for the full-data presence run of Section 5.4.3, which uses the identical optimizer configuration, and it turns out to say something the loss curves do not. Across the 63 logged points the gradient norm ranges from 0.55 to 11.97 with a median of 4.66, and **98.4% of logged steps exceed the clipping threshold of 1.0**. Nothing diverged (there are no NaNs, no spikes into the hundreds, and the

loss falls smoothly throughout) so the runs were healthy in the sense that matters. But clipping here is not the rare safety net the phrase “gradient clipping” usually implies. It is active on essentially every step, which means the effective update direction is being renormalized continuously and the nominal learning rate of 2×10^{-5} overstates the step size actually taken. We report this because it slightly weakens the warmup argument made above: part of what kept the early updates stable was almost certainly the clipping rather than the small learning rate, and we would not want a reader to take “no warmup was fine” as a general recommendation on the strength of our smooth loss curves alone. Re-running the span and summarizer trainings with the same logging would close the remaining gap in about half an hour of wall-clock time.

4.4.3 Training the Presence Classifier

The TF-IDF + logistic-regression baseline trains in under a minute. On the 20% test set it reaches micro-F1 0.775, weighted-F1 0.777, macro-F1 0.572 over all 41 categories, and 0.666 over the 34 categories that have at least ten test positives. The gap between micro and macro comes from the near-few-shot tail of the dataset, not from a modelling failure.

The window-level DistilBERT was trained on a stratified subsample of 24,000 window examples (keeping the positive rate at 39.2%) so the run stays tractable on a laptop. Training and validation losses fell smoothly ($0.320 \rightarrow 0.261$ train, $0.278 \rightarrow 0.257$ validation over two epochs; Figure 4.10), and the two curves stayed close together, so we saw no sign of overfitting at this scale.

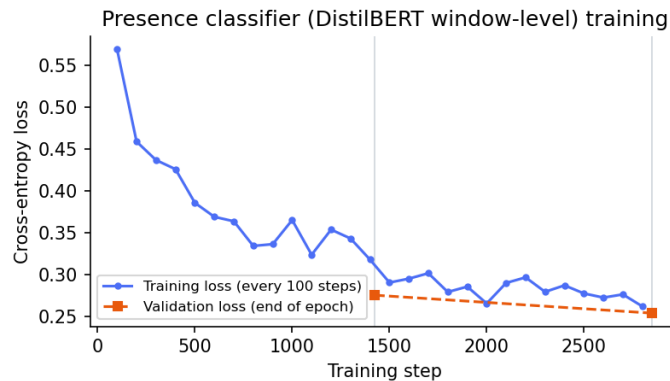


Figure 4.10: Presence classifier loss

4.4.4 Training the Span Extractor

The QA model trained on 8,000 focused-window examples for two epochs. Before training we decode the token-labelled spans back into text for a few examples and check they match the gold answers: SQuAD-style pipelines make it very easy to get the character-to-token offset conversion silently wrong, and this small sanity check catches that.

4.4.5 Training the Summarizer

FLAN-T5-small was fine-tuned on 4,000 clause-template pairs on CPU: about 8 minutes originally, and 12.7 minutes in the October 2026 re-training (run while the machine was busy with other jobs). The training loss collapsed quickly, from 1.17 at step 100 to 0.08 by the end of the second epoch (Figure 4.11); there is no validation curve because the deployed configuration uses no evaluation set (Table 4.4). The re-trained model scores ROUGE-L 0.775 against the templates on the 150 test clauses, identical to the original. In hindsight this rapid collapse was the first hint of the template-memorization behaviour we analyse in Chapter 5: with only 41 unique targets, the task is simply easy to fit.

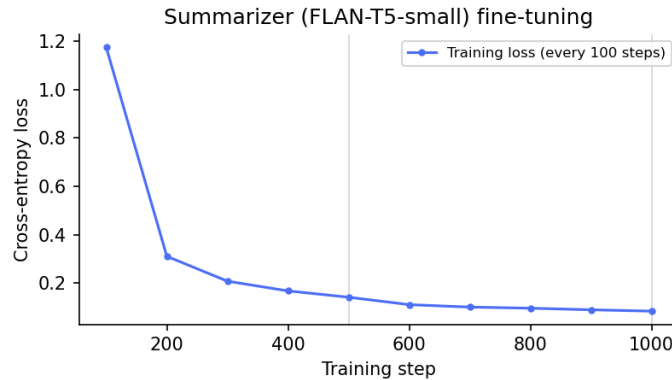


Figure 4.11: Summarizer loss

4.4.6 Engineering Notes: Training on Consumer Hardware

A few practical findings may save time for anyone reproducing this work on Apple-Silicon machines:

- **DeBERTa-v3 is unstable on MPS.** Our first-choice backbone produced NaN gradients on the MPS backend and was far too slow on CPU (about 89 seconds per step). We switched to DistilBERT; the method is unchanged, and on CUDA hardware the backbone string can simply be swapped back.
- **MPS memory does not release in-process.** After two transformer trainings in the same session, the summarizer hit the MPS memory ceiling even after we deleted objects and emptied the cache. The fixes that actually work are restarting the kernel or training the (small) summarizer on CPU, which is what we did.
- **API migration.** `transformers` 5.x renamed `Trainer(tokenizer=...)` to `processing_class=...`. We also disabled mixed precision (`fp16=bf16=False`) and cast loss logits to `fp32` for MPS dtype safety.
- **Reproducibility.** Every artifact (the split in `artifacts/split.json`, the fitted baseline in `artifacts/baseline.pkl`, and the three model checkpoints) is saved to disk, and a standalone evaluation notebook regenerates every number in Chapter 5 from those files alone.

4.4.7 Measured: What Sparse Attention Would Have Cost

Section 2.1.2 argued against sparse attention partly on cost, and an argument from cost that never measures the cost is not worth much. Since the machine used for this project is the machine in question, the measurement is available for the price of running it, so we ran it.

We instantiated Longformer-base at its published geometry (12 layers, 768 hidden units, 4,098 position embeddings, attention window 512, 148.7M parameters) and timed a forward pass at 4,096 tokens and a full training step (forward, backward, AdamW update) against the DistilBERT configuration this project actually used. Weights are randomly initialized, which is legitimate here because time and memory depend on tensor shapes and not on the values in them. Each configuration ran in its own process so the peak Metal allocation reported belongs to that configuration alone. The machine is an Apple M4 with 16 GB of unified memory, shared with the operating system. Table 4.5 reports the result.

Table 4.5: Longformer vs. DistilBERT cost

Model	Operation	Batch	Time	Peak memory
Longformer-base (148.7M)	forward, 4,096 tok	1	3.69 s	2.18 GB
DistilBERT (67.0M)	forward, 256 tok	16	0.19 s	1.15 GB
Longformer-base	training step, 4,096 tok	1	258 s	14.71 GB
Longformer-base	training step, 4,096 tok	2	2,170 s	20.75 GB
DistilBERT	training step, 256 tok	16	3.17 s	4.47 GB

Note. Measured on the Apple M4 (16 GB unified memory) that trained every model reported in this project.

The first thing this measurement does is correct us. Section 2.1.2’s original wording claimed that fine-tuning a Longformer at 4,096 tokens was “far beyond a single laptop’s memory budget”. That is not what happens. At batch size 1 it fits: 14.71 GB on a 16 GB machine. It is uncomfortably close to the ceiling, that figure leaves under a gigabyte and a half for the operating system, but the model, its gradients and its optimizer state all sit in memory and the step completes. We had asserted an impossibility that turns out to be a tight fit, and the assertion was wrong.

Where the memory claim does hold is one step further out. At batch size 2 the requirement rises to 20.75 GB, past the physical memory of the machine, and the consequence is visible in the timing: the step takes 2,170 seconds rather than the roughly 516 that doubling the batch would predict: 8.4 times slower for twice the work, which is the signature of paging rather than computing. So the honest form of the claim is not “it does not fit” but “it fits only at batch size 1, and any batch size a real training run would use does not”.

The binding constraint, though, is time, not memory. At 258 seconds per step and batch size 1, reproducing our two-epoch presence schedule (22,800 training windows, 45,600 sample presentations) would take roughly **136 days** of continuous compute. The DistilBERT run it replaces took 47 minutes. Even granting a generous allowance for kernel-compilation overhead in a short benchmark (our own

DistilBERT step measures 3.17s here against about 1.0s sustained in the real run, so these figures should be read as upper bounds inflated by roughly threefold), the conclusion does not become sensitive to that correction: a third of 136 days is still 45 days, and the project had weeks, not months.

The inference-side objection also survives contact with a stopwatch. Scanning one median contract (33,143 characters) for all 41 categories takes $22 \text{ windows} \times 41 \times 11.7 \text{ ms} \approx \mathbf{10.6 \text{ seconds}}$ with our DistilBERT, against $3 \text{ windows} \times 41 \times 3.69 \text{ s} \approx \mathbf{454 \text{ seconds}}$, seven and a half minutes, with Longformer. Clausify’s design target is a response in seconds, and this is the measurement that rules the alternative out for the deployed system regardless of what a training cluster could manage.

None of this establishes that a Longformer would have scored worse; it establishes only that we could not have found out on this hardware, which is a different and much narrower claim than the one we started with. The comparison that would settle the modelling question needs a CUDA machine, and we list it in future work rather than pretending the cost argument answers it.

4.4.8 End-to-End Data Flow

Figure 4.1 names the stages; Figure 4.12 adds what actually moves between them. The shapes shown are for one contract of median length passing through the deployed system.

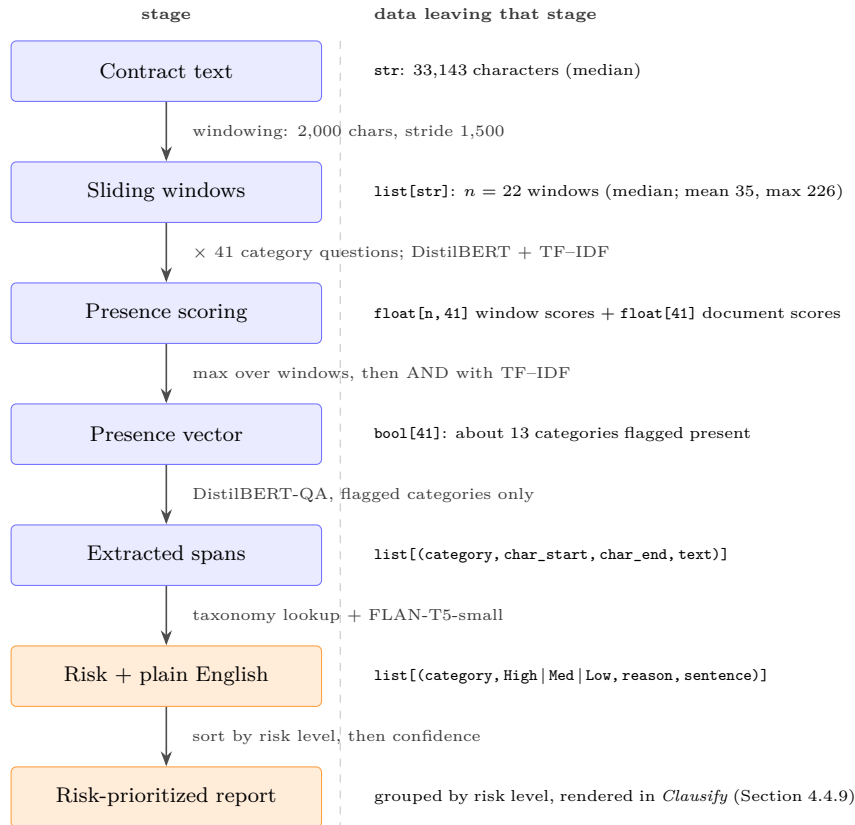


Figure 4.12: Pipeline data flow

4.4.9 The Clausify Application

Purpose and Design

The final deliverable is *Clausify*, a Streamlit web application that runs all of the project’s trained models live on any contract the user provides (the DistilBERT presence classifier, the TF–IDF baseline for the optional AND-ensemble, the span extractor and the FLAN-T5 summarizer) and presents the findings as a risk-prioritized report: the pipeline of Figure 4.1, made usable. The design goal is simple: a non-lawyer should see the “watch out before signing” items first. Detected clauses are grouped **High** → **Medium** → **Low** and sorted by model confidence within each group.

User Workflow

1. **Add a contract.** Paste the text, or upload a `.pdf`, `.docx` or `.txt` file; text is extracted locally (Figure 4.13). The user chooses *Whole contract*, *Separate clauses* (Section 4.4.9) or *Auto*; a settings panel selects the detection rule: DistilBERT alone (default) or the AND-ensemble; and turns the plain-English lines on or off. Detection uses the fixed 0.5 threshold of the reported evaluation.
2. **Analyze.** One click runs the models end to end; a progress bar tracks the scan across the 41 categories and then the summarizer.
3. **Read the report.** A summary bar counts the detected clauses per risk level (Figure 4.15), and below it each detected clause appears as a card. On the synthetic test contract the bar reports 7 High, 14 Medium and 9 Low risk clauses: 30 of the 41 categories detected (October 2026 models, DistilBERT detection).

Each clause card shows the category name with a colour-coded risk badge; the one-line reason that category carries its risk (from the taxonomy in Appendix A); the summarizer’s plain-English sentence, flagged when it is another category’s template; the detection score drawn as a bar, with both the DistilBERT and TF–IDF scores; and the located clause quoted from the user’s own contract with the span model’s key phrase highlighted (Figure 4.16). Below the cards the application adds three views:

- **Highlighted contract.** The full text, with every located clause marked in its risk colour and tagged with its category; each card links to its place in the text.
- **Missing-protection checklist.** Protective clauses the models did *not* detect, namely a cap on liability, termination for convenience, governing law, an expiration date, a warranty period, insurance and, when the contract renews, a notice period to stop renewal, each with a sentence on why its absence matters. The view states that “not detected” can be a model miss.

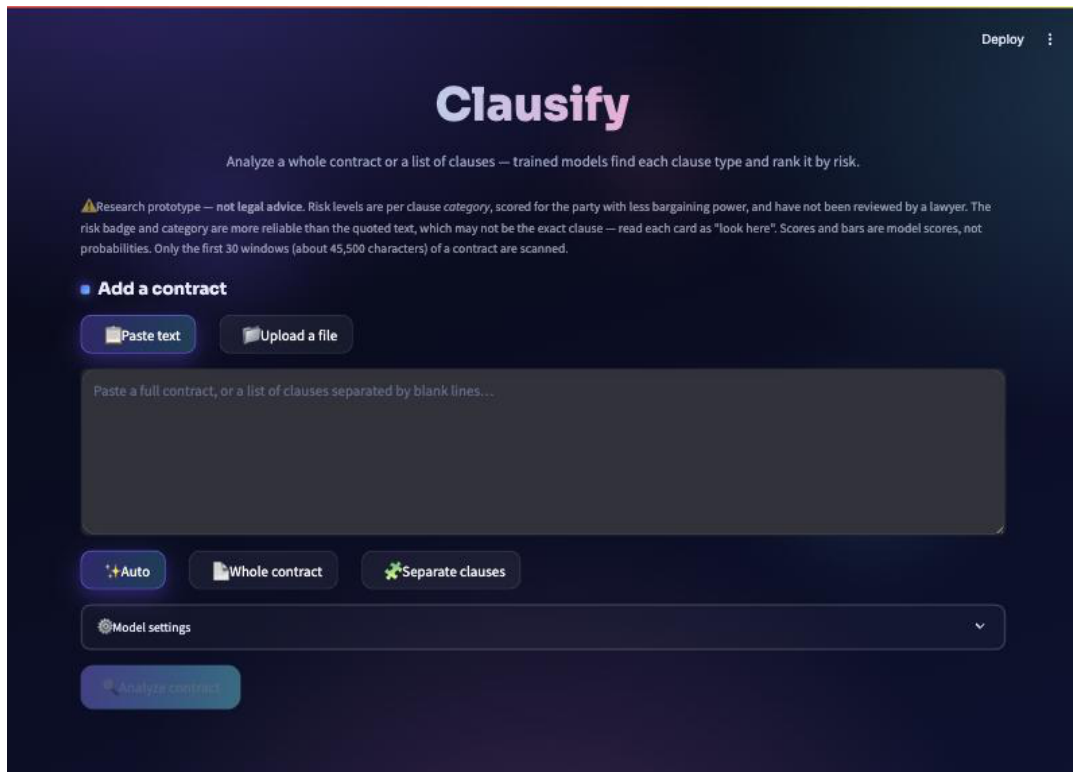


Figure 4.13: Clausify input screen

- **Downloadable report.** A PDF report (summary, every clause with its reason, plain-English line, scores and quoted text, and the checklist) and a CSV of the same rows.

Figure 4.14 shows the highlighted view and the checklist on the synthetic test contract. The renewal paragraph tagged *Minimum Commitment* there is a genuine locating error by the model, left in deliberately rather than replaced by a cleaner example.

How the Models Run Inside the App

The application reproduces the inference path from training exactly:

1. **Windowing.** The contract is cut into 2,000-character windows with a 1,500-character stride, the same scheme used in training, capped at 30 windows so very long documents stay responsive.
2. **Model 1: presence.** For each of the 41 categories, the exact CUAD question the model was trained on is paired with every window and scored by the window-level DistilBERT. The per-category document score is the max-pooled window probability; categories above the threshold are reported present, and the best-scoring window is remembered.
3. **Locating the clause.** Within that best window, the presence model scores each paragraph against the category question and the highest-scoring paragraph is the text quoted on the card; the DistilBERT-QA span model then

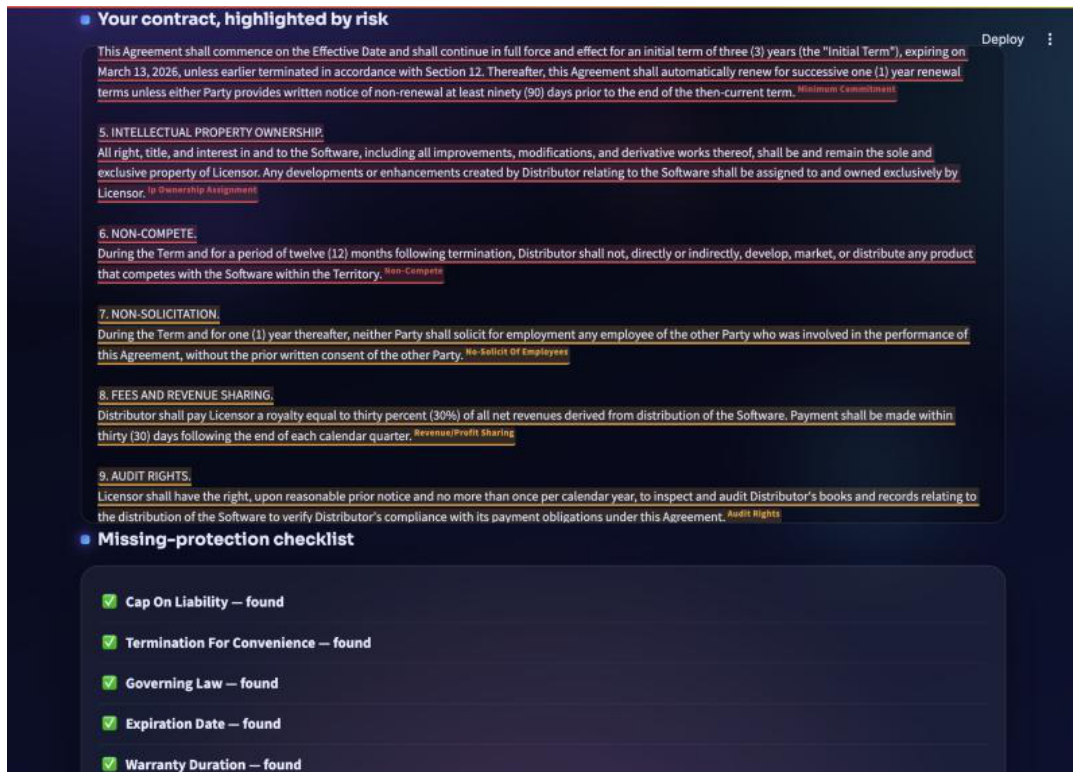


Figure 4.14: Highlighted contract and checklist

highlights the key span inside it. An earlier version let the span model choose the text from the whole window, and on test inputs it returned the same sentence for nearly every category: the span model favours text near the position where answers sat during training (Section 5.2.1), so it is used only inside the paragraph the presence model selected.

4. **Optional AND-ensemble.** With the ensemble setting, the TF-IDF baseline scores the whole contract and a category is reported only when both models' scores reach 0.5, exactly as evaluated in Section 5.4.1.
5. **Risk layer.** The detected category is looked up in the fixed 8/22/11 taxonomy for its level and reason.
6. **Summarizer.** FLAN-T5-small receives each quoted clause with the training prompt and decoding settings and returns one plain-English sentence.

All inference runs on CPU so the app works on any machine; a median-length contract takes about 27 seconds once the models are loaded (Table 4.6). The app loads the actual trained checkpoints saved by the training pipeline (`presence_mil`, `span`, `summarizer` and the TF-IDF `baseline.pkl`); when they are not present locally, as on a cloud host, it downloads the same files once from a public Hugging Face model repository.

Implementation Notes

The interface is a single-page Streamlit application (`app.py`) sitting on a small inference module (`model_utils.py`) that owns model loading, windowing, max-pooling,

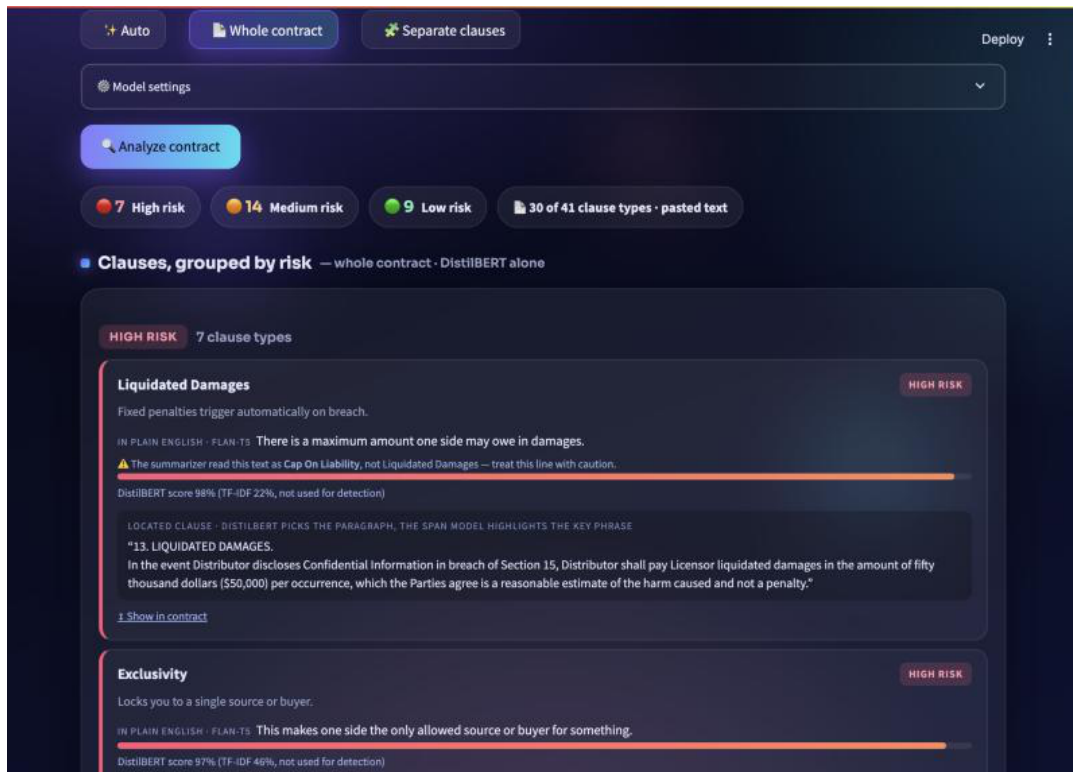


Figure 4.15: Clausify results view

span extraction, and the risk table. Two test contracts ship with the app: a genuine CUAD contract, and a synthetic “Master Software License and Distribution Agreement” that we wrote specifically to exercise a wide spread of the 41 categories. On that synthetic contract the app detects clauses across all three risk bands and puts *Uncapped Liability*, *IP Ownership Assignment*, *Exclusivity*, and *Liquidated Damages* at the top of the report: exactly the before-you-sign warnings the design aims for.

Clause-by-Clause Mode

Users often paste a handful of clauses rather than a whole contract, and expect one categorized card per clause. Whole-contract mode does not do this: it asks, for each of the 41 categories, whether the category appears anywhere in the window, so six related clauses pasted together produced fourteen detected categories, eight of them spurious. The application therefore offers a second mode (chosen automatically for a short list of paragraphs, or selected by the user) in which each paragraph is one clause and receives exactly one category and one risk level.

Within this mode a clause heading that names a category (“2. EXCLUSIVITY.”) decides the category directly; headings that do not unambiguously name one category are ignored. Otherwise the presence model scores the clause against all 41 category questions. Raw scores cannot simply be compared across categories (*Parties*, for example, scores about 0.98 on almost any clause that names the two sides) so each category’s log-odds is standardized against how that category scores on genuine CUAD clauses of *other* categories, using 1,586 clauses from the training split, and the clause takes the category with the highest standardized score. A clause whose



Figure 4.16: High-risk clause cards

best standardized score is below 2 is shown as *unrecognized* rather than forced into a category.

Measured on 721 clauses from the *test* split, this calibration raises single-clause accuracy from 9.6% (raw arg-max) to **42.3%** over 41 categories; the correct category is in the top three 65.3% of the time, and the correct risk level is assigned 67.0% of the time. That is well below the document-level presence scores, which is expected (the presence model was trained to scan 2,000-character windows, not to label isolated clauses) and the application states these numbers under every clause-mode result. On the six-clause example above, the headed version is categorized correctly (five High, one Low); with the headings removed the model alone gets none of the six risk levels right (it labels five of them Medium and leaves one unrecognized), so on inputs like this the clause headings carry the accuracy and model-only cards should be checked against the clause text.

Measured Latency

Chapter 2 justified the choice of a distilled backbone partly on the grounds that the interface must respond quickly. Section 4.4.7 measured what the alternative would have cost; this section measures what *we* actually deliver, running the application’s own inference path on held-out contracts spanning the length distribution.

An earlier draft claimed a contract takes “roughly 5–15 seconds”. That was an impression from watching the demo on short inputs, not a measurement, and it was wrong: a median-length contract takes 27.1 seconds.

Table 4.6: Application latency

Contract	Characters	Windows scored	Time	Peak RSS
Shortest in test set	1,283	1	2.1 s	1.06 GB
25th percentile	18,104	12	13.3 s	1.19 GB
Median	35,136	24	27.1 s	1.50 GB
75th percentile	65,547	30 (capped)	32.4 s	1.51 GB
Longest in test set	289,615	30 (capped)	32.0 s	1.52 GB
<i>median 27.1 s mean 21.4 s worst case 32.4 s</i>				

Note. Measured through the application’s own `analyze()` entry point. CPU only, Apple M4, models already loaded (cold load adds 0.4 s). Peak RSS is the whole Python process, both models resident.

Two design consequences are visible in the numbers. First, latency is essentially linear in the number of windows, since the dominant cost is 41 categories $\times$ windows forward passes through DistilBERT: the model is scored once per (category, window) pair and nothing is cached across categories. Second, the 30-window cap is what keeps the worst case bounded: without it the longest test contract would take several minutes, and with it every contract above roughly 47,000 characters receives the same truncated treatment. That cap is a latency decision with a silent recall cost, clauses beyond the cap cannot be found at all, and it is not reflected anywhere in the Chapter 5 numbers, which score the full document. It is a limitation of the deployed system rather than of the method, and Section 5.6.2 now says so.

Chapter 5

Result Analysis

5.1 Performance Evaluation

Evaluation criteria. Each stage is measured with the metric that matches its task. *Presence classification* is a binary decision for each of the 4,182 (contract, category) test pairs; it is evaluated by micro-F1 (every decision weighted equally), macro-F1 (every category weighted equally), and the accuracy, precision, recall and specificity derived from the confusion matrix. *Span extraction* is evaluated by token-level F1, exact match and the share of predictions overlapping the gold clause at $F1 \geq 0.5$. *Summarization* is evaluated by ROUGE-L against two different reference sets. The *complete pipeline* is evaluated by the share of gold clauses that survive every stage, and the risk layer by the recall of clauses in each risk band. Efficiency is measured as end-to-end latency and memory of the deployed application (Table 4.6).

Testing methods. All models are tested once on the same 102 held-out contracts, using a standalone notebook that loads the saved checkpoints from disk. Uncertainty is quantified with a cluster bootstrap over contracts, and the robustness of the main comparison is tested against training budget, random seed, aggregation rule, decision threshold and dataset split.

Everything in this chapter is measured on the held-out 20% test set: the same 102 unseen contracts for every model. The numbers were regenerated by a standalone evaluation notebook that loads the trained models from disk, so no state left over from the training session can influence them.

5.1.1 Presence Classification

Model comparison

Table 5.1 and Figure 5.1 compare all the presence models on the 4,182 (contract, category) test decisions. Two things stand out. First, the windowed transformer *loses on its own* to the full-document TF-IDF baseline (0.694 vs. 0.775). Legal vocabulary is distinctive enough that lexical evidence over the whole document beats semantic evidence over windows, and max-pooling inflates false positives exactly the way Section 4.2.1 predicted. Second, the AND-ensemble, which reports a clause only

when *both* models agree, filters out each model’s uncorrelated false positives and finishes on top (0.776). The margin over the baseline is small on 102 test contracts, so we do not lean on it: Section 5.4.1 quantifies exactly how small, and finds that this particular gap is indistinguishable from zero. The conclusion we stand behind is methodological: *simple lexical models are strong at document-level presence, and the transformer earns its place as a precision filter, not as a replacement.*

Table 5.1: Presence model comparison

Model	micro-F1
Ensemble AND (both must agree)	0.776
TF-IDF baseline (full document)	0.775
Ensemble AVG (mean probability)	0.718
Ensemble OR (either fires)	0.696
Transformer (max-pool) alone	0.694
<i>control run, Section 5.4.3:</i>	
Transformer trained on all 53,662 windows	0.694
Ensemble AND on that transformer	0.782

Note. micro-F1 over all 4,182 test cells.

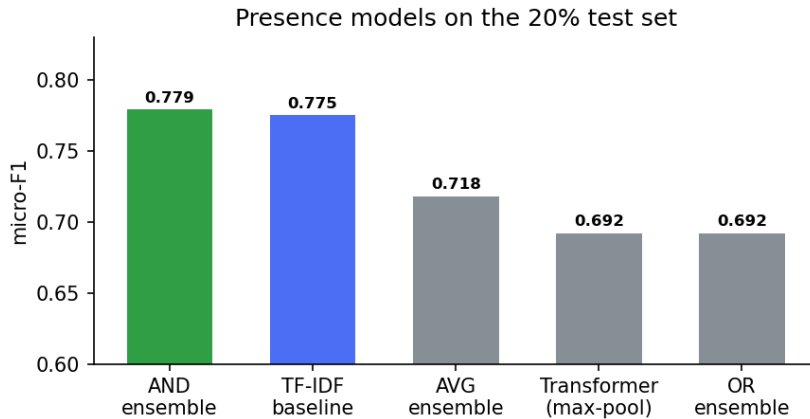


Figure 5.1: Presence model micro-F1

Confusion-matrix analysis

Our evaluation protocol called for a full confusion-matrix analysis of the final model. Table 5.2 gives the matrix for the AND-ensemble over all test decisions, Figure 5.2 shows the same matrix as a heatmap, and Table 5.3 lists every metric derived from it.

Precision and recall come out balanced (77.2% and 78.0%), which tells us the AND rule did its job: raw max-pooling gives up precision, and requiring agreement wins it back without sacrificing recall. The specificity of 89.0% means the system rarely claims a clause that is not there: in a legal-review assistant, that is the more damaging kind of mistake.

Table 5.2: AND-ensemble confusion matrix

	Predicted Absent	Predicted Present
Actually Absent	TN = 2,523	FP = 311
Actually Present	FN = 296	TP = 1,052

Table 5.3: Confusion-matrix metrics

Metric	Formula	Value
Accuracy	$(TP + TN)/\text{all}$	85.49%
Precision	$TP/(TP + FP)$	77.18%
Recall (Sensitivity)	$TP/(TP + FN)$	78.04%
Specificity	$TN/(TN + FP)$	89.03%
F1 score	$2PR/(P + R)$	0.7761

Per-category behaviour

Figure 5.3 shows per-category F1 for the strongest categories. The frequent, formulaic ones are close to solved (*Document Name* 1.000, *Parties* 0.995, *Agreement Date* 0.963, *Governing Law* 0.931). At the other end, five categories with at most seven test positives score an F1 of 0.000 under the AND-ensemble. Table 5.4 shows exactly what that number means. In every case the ensemble predicted the category in *no* test contract (no true and no false positives), so F1 = 0.000 is correct, none of the 2–7 real clauses was found, but precision is undefined (0/0) rather than zero. The transformer on its own does find some of these clauses; the conjunction removes those detections because the TF-IDF model never agrees (Section 5.2.4). With two to seven positives per category these scores are not reliable estimates in either direction. This tail is a property of the dataset rather than of any architecture, and it is why we quote micro- rather than macro-averaged numbers in our headline results.

Table 5.4: Categories with zero F1

Category	Test pos.	AND: TP / FN	AND precision	Transformer alone F1 (TP/FP/FN)	TF-IDF F1
Most Favored Nation	7	0 / 7	n/a	0.000 (0/0/7)	0.000
No-Solicit of Customers	6	0 / 6	n/a	0.323 (5/20/1)	0.000
Price Restrictions	4	0 / 4	n/a	0.087 (1/18/3)	0.000
Source Code Escrow	2	0 / 2	n/a	0.000 (0/0/2)	0.000
Unlimited/All-You-Can-Eat License	2	0 / 2	n/a	0.500 (1/1/1)	0.000

Note. n/a: the model made no positive predictions, so precision = 0/0 is undefined. Threshold 0.5, October 2026 models.

5.1.2 Span Extraction

We evaluated the QA model on all 2,667 answer-bearing test windows (Table 5.5). A token-F1 of 0.763, with 82.7% of predictions overlapping the gold span at F1 ≥ 0.5 , means that when the clause is in front of the model, it captures most of the right text roughly eight times out of ten. The lower exact-match score did not surprise us: annotated legal spans are long, and their exact boundaries are genuinely

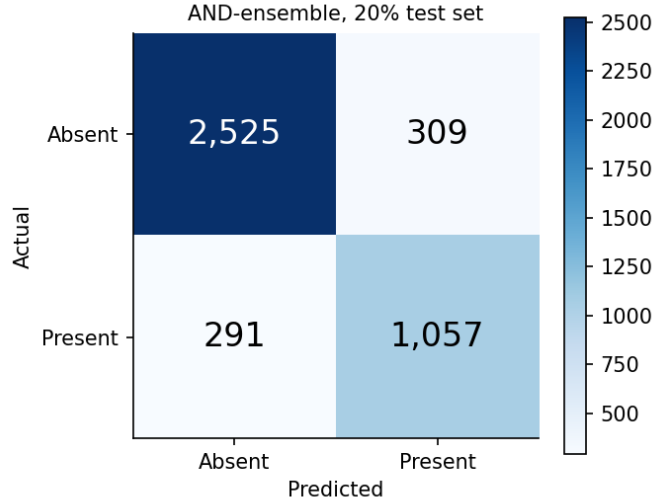


Figure 5.2: Confusion matrix heatmap

debatable (should a leading section number be included?), so content overlap is the metric that carries information here. For scale, the best published CUAD result uses DeBERTa-xlarge, roughly an order of magnitude more parameters than our 67M-parameter distilled backbone.

Section 5.4.1 argued that a point estimate on 102 contracts should not be read without an interval, and that argument does not stop applying when the task changes. We therefore ran the *same* cluster bootstrap here. The correlation structure is if anything stronger than in presence classification: the 2,667 span decisions come from only 102 contracts, and a contract with unusual drafting conventions (numbered sub-clauses, defined-term-heavy prose) shifts every span in it in the same direction. Table 5.5 therefore reports two intervals per metric: one resampling *contracts* (the correct unit) and one resampling *clauses* independently (the naive unit), because the gap between them is itself informative.

Table 5.5: Span extraction results

Metric	Score	95% CI (by contract)	95% CI (by clause)
Token-level F1	0.763	[0.740, 0.782]	[0.751, 0.774]
Overlap (token-F1 ≥ 0.5)	82.7%	[79.7%, 85.1%]	[81.3%, 84.1%]
Exact Match	35.5%	[32.5%, 38.6%]	[33.7%, 37.3%]

Note. 95% bootstrap intervals, 10,000 resamples. The contract-clustered interval is the one to quote; the clause-level interval is shown only to expose how far a naive resampling unit understates the uncertainty.

The contract-clustered interval on token-F1 is [0.740, 0.782]: roughly *twice* as wide as the [0.751, 0.774] that treating clauses as independent would have produced. That factor is the practical cost of ignoring clustering, and it is the same effect that made the cluster bootstrap necessary in Section 5.4.1. It also sets the resolution of this number: differences below about two points of token-F1 are not measurable on a test set of this size, which is worth remembering before comparing 0.763 against any

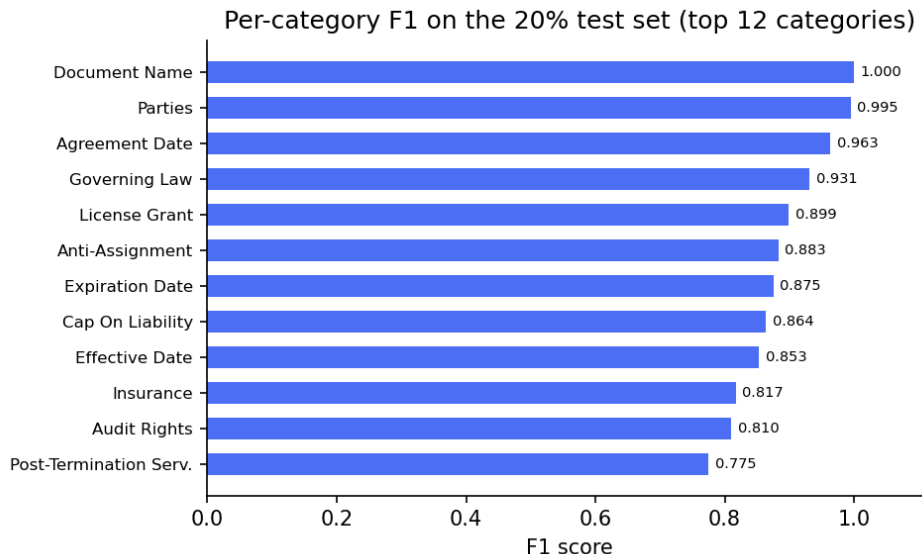


Figure 5.3: Top-12 per-category F1

published figure.

Scope caveat. These numbers measure extraction quality *given an answer-bearing window*. More precisely, each test window is the 1,200-character focus window re-centred on the gold answer (Section 4.2.5), so the model is told, implicitly, where the answer is; the benchmark is an oracle upper bound on extraction, not a deployable score. The full-document benchmark in the CUAD paper (precision at 80% recall over all windows, including the answer-free ones) needs a heavier document-stride ranking evaluation that we did not run, so we make no claim on that metric. Section 5.2.1 measures the complementary thing: what happens to span quality when the window is chosen by the presence stage instead of being handed over.

5.1.3 Summarization

What the reference set actually is

Before any number in this section can be read correctly, one property of the experiment has to be stated plainly, because it determines what the metric can possibly mean. CUAD contains no plain-English rewrites, so there was no annotated target set to train or test against. We therefore **wrote the targets ourselves**: 41 single-sentence summaries, one per category, phrased from the weaker party’s perspective (Section 4.2.3). Every training pair and every evaluation pair reuses one of those 41 sentences. There is no clause-level variation in the target set at all: two *Cap on Liability* clauses with completely different caps share a target string, character for character.

That is a design choice with a consequence we should own rather than discover: with 41 unique targets covering 2,667 test clauses, a model can score well by learning to *recognise* the category and reproduce its sentence, and the metric cannot tell that apart from summarization. The experiment below is built to separate the two.

Two reference sets, the same 150 clauses

We evaluate the fine-tuned FLAN-T5-small on 150 held-out test clauses (sampled with seed 42) against two different reference sets:

1. **Template references:** the category template, exactly as in training. This is the protocol behind the headline number.
2. **Clause-specific references:** 150 one-sentence summaries, one per clause, written to describe *that* clause: its actual cap, its actual notice period, its actual parties. These were drafted with a large-language-model assistant working from the clause text alone, without sight of any model output, and are deliberately independent of the 41 templates. They are a pilot reference set, not lawyer-authored annotation, and we make no claim beyond that.

Because the clauses are identical across the two conditions, the difference between the two scores isolates exactly one thing: how much of the reported ROUGE-L comes from reference-set uniformity rather than from summarization. Table 5.6 gives the result.

Table 5.6: Summarizer ROUGE-L

System	Reference set	ROUGE-L	95% CI
Fine-tuned FLAN-T5-small	41 category templates	0.775	[0.718, 0.833]
Fine-tuned FLAN-T5-small	clause-specific (pilot)	0.116	[0.105, 0.128]
Zero-shot FLAN-T5-small	clause-specific (pilot)	0.299	[0.275, 0.322]
<i>Category template itself</i>	clause-specific (pilot)	0.115	[0.104, 0.127]

Note. Intervals are 95% bootstrap CIs over the 150 clauses (10,000 resamples).

Reading the table

Three things follow, and none of them is a hedge.

The 0.775 is template reproduction, and we can now say so quantitatively. On all 150 test clauses the fine-tuned model emits one of the 41 template sentences *verbatim*: 100% of outputs are an exact copy of a template, the correct one 70% of the time and a different category’s template the other 30%. The model never produces a sentence of its own. When it is wrong it is wrong in the way a classifier is wrong: a *Price Restrictions* clause about a 10% rate cap is answered with the *ROFR* template. Fine-tuning did not teach the model to summarize; it taught it to pick one of 41 strings.

The last row of Table 5.6 closes the argument. The bare category template, with no model involved at all, scores 0.115 against the clause-specific references: statistically indistinguishable from the fine-tuned model’s 0.116. A constant lookup table achieves the model’s entire score. Whatever the fine-tuned summarizer has learned adds nothing measurable on top of “print this category’s sentence”.

Fine-tuning made the model worse at the actual task. Against clause-specific references the *zero-shot* model scores 0.299, more than twice the fine-tuned model’s 0.116. We do not present this as evidence that zero-shot FLAN-T5 is a good legal summarizer: it is not; as Table 5.7 shows, it largely echoes the source text, and echoing inflates a longest-common-subsequence metric against a reference that necessarily shares the clause’s vocabulary. The honest reading is narrower and still uncomfortable: our fine-tuning traded away whatever clause-specific content the base model had, in exchange for fluent category-level boilerplate that scores well only against a reference set built from the same boilerplate.

Table 5.7: Fine-tuned vs. zero-shot output

Input clause	“Any attempted assignment or delegation in violation of this section by either party without the prior written consent of the other will be void.”
Gold template	You cannot transfer this contract to someone else without permission.
Fine-tuned	You cannot transfer this contract to someone else without permission. (<i>exact template</i>)
Zero-shot	“a) Any attempted assignment or delegation in violation of this section ...will be void.” (<i>echoes the legalese</i>)
Clause-specific reference	Neither side may hand its rights or duties to a third party without the other’s written consent, which cannot be unreasonably refused.

What we therefore claim

We claim a working **classification-to-template** system: it identifies a clause’s category and attaches a pre-written, human-authored plain-English sentence for that category, which would be genuinely useful in an interface: a signer reading “You cannot transfer this contract to someone else without permission” next to a wall of legalese is better off than before. (Clausify shows this line on every card, labelled as the summarizer’s output and placed beside the fixed per-category reason from the risk taxonomy; when the sentence is another category’s template, the card says so; Section 4.4.9.) We do **not** claim free-text legal summarization, and the 0.775 should never be quoted as evidence of it. The number that describes clause-level summarization ability is 0.116, and the honest conclusion is that this component is a proof of concept, not a result.

Getting past this needs a target set with clause-level variation, which is exactly what the pilot above provides a small taste of: 150 references took a few hours to draft. Section 6.5 sets out the scaled version.

5.2 Analysis of Design Solutions

This section analyses the selected design as a whole: what survives the complete pipeline, whether its errors fall where they matter for a signer, what the errors look

like, and how the ensemble behaves on rare categories.

5.2.1 End-to-End: What Actually Reaches the User

Every number so far has been measured with the other stages held out of the way. Presence classification was scored given the ground-truth category list; span extraction was scored given a window already known to contain the answer; summarization was scored given a gold clause. Each of those is the right way to evaluate a component in isolation, and none of them describes what the pipeline does to a contract. The stages run in series, and errors compound: a category the presence stage misses is a clause the span stage never sees.

So we ran the pipeline end to end on all 102 test contracts and asked one question of each of the 1,348 gold clauses: does it survive *all three* stages? A clause counts as surviving only if (i) the presence stage detects its category, (ii) the span stage (reading the window the presence stage selected, never a window chosen using the gold offset) returns text overlapping the gold clause at token-F1 ≥ 0.5 , and (iii) the risk level attached is the correct one. Table 5.8 gives the attrition.

Table 5.8: End-to-end clause survival

Stage	Condition	Clauses surviving	Share of gold
–	gold clauses in the test set	1,348	100.0%
2A Presence	category detected	1,052	78.0%
2B Span	extracted text overlaps gold at F1 ≥ 0.5	293	21.7%
Risk	correct level attached	293	21.7%
End-to-end survival		293	21.7%
<i>95% CI, cluster bootstrap over contracts</i>			[19.6%, 23.9%]

Note. AND-ensemble presence rule. The last column is the fraction of clauses still alive after that stage.

21.7% is the number that describes a user’s experience, and it is far below what the component scores would lead a reader to expect. Multiplying the isolated figures, 78.0% presence recall times 82.7% span overlap, predicts about 64%. The pipeline delivers a third of that. The gap is the whole point of running this evaluation, so it deserves to be explained rather than reported.

The risk stage is the one place where nothing is lost: the level is a fixed per-category lookup, so it is correct exactly when the category is, and it introduces no error of its own. That is a property of the design, not an achievement, and it is worth remembering that “correct” here means “matches our taxonomy”, which Section 4.2.4 notes has not been externally validated.

Where the clauses are lost

The collapse happens at stage 2B, and it has two distinct causes, which we separated by checking, for every detected clause, whether the window the presence model selected actually contained the gold text.

Table 5.9: Span-stage loss breakdown

	Count	Share
Selected window <i>contains</i> the gold clause	718	68.3%
, and the span model then found it ($F1 \geq 0.5$)	285	39.7% of 718
, and the span model missed it anyway	433	60.3% of 718
Selected window does <i>not</i> contain the gold clause	334	31.7%
: span scored ≥ 0.5 regardless (duplicate wording elsewhere)	8	

Cause one: the max-pooled window is often the wrong window. Max-pooling was chosen (Section 4.2.1) because it removes the recall ceiling (if any window fires, the category is reported) and Section 4.2.1 was careful about what that costs in *precision*. What it did not anticipate is that the argmax window is also being used as a *locator*, and it is a poor one: 31.7% of the time the highest-scoring window does not contain the clause at all. The MIL objective never asked it to. The model was trained to answer “does this window contain a clause of this category”, and a window that discusses liability in general terms can outscore the one that actually caps it. For those 334 clauses the span stage is being handed the wrong 2,000 characters, and no extractor could recover them.

Cause two: the span model is much weaker on windows it was not trained to see. Even with a window that does contain the clause, extraction succeeds only 39.7% of the time, against 82.7% in the isolated benchmark of Section 5.1.2. The reason is the focus-window construction of Section 4.2.2. At training time every example was re-centred so the answer began roughly 150 characters into a 1,200-character window. That is legitimate: it uses a label to build a training example, never an inference input, and Section 4.2.5 defends it on exactly those grounds. But legitimacy is not the same as harmlessness: the model has only ever read windows in which the answer sits near the start, and at inference it gets blind 1,200-character chunks in which the answer may be anywhere. Mean token-F1 falls from 0.763 to 0.386. The re-centring did not leak the test set; it did quietly narrow the distribution the model can handle.

What this changes

Three things follow, and we would rather state them than let the component tables stand unqualified.

First, **the span numbers in Section 5.1.2 are an upper bound on deployed extraction quality, not a description of it.** They answer “how good is the extractor when handed the right window”, which is a fair question about the model and a misleading one about the system.

Second, **the most valuable next engineering step is not a bigger backbone.** Cause two is a data-construction problem with a cheap fix (train the span model on windows where the answer is placed at varied offsets, or on the same blind chunks it will see at inference) and cause one argues for choosing the extraction window by a locator trained for that job rather than reusing a presence score. Both are within the compute budget of this project; neither requires more parameters.

Third, **Clausify’s output should be read as a shortlist, not an extraction.** When the application shows a clause card, the category is right about 78% of the time it was present, but the quoted text is a good rendering of the actual clause in roughly one case in five overall. The application tells users it is not legal advice, and on this evidence it also tells them that the quoted text may not be the clause the badge refers to (Section 5.6.2).

5.2.2 Risk-Weighted Evaluation: Does the Metric Match the Purpose?

Micro-F1 over 4,182 decisions treats every decision as equally important. A contract signer does not. The whole premise of the risk layer (Section 4.2.4) is that some categories matter more than others, so the evaluation should ask whether the system’s errors fall where they do least harm. We had not asked, and the answer turns out to reverse the recommendation the aggregate produces.

Table 5.10 splits the 41 categories into the three risk bands of our own taxonomy and scores each model within each band.

Table 5.10: Results by risk band

Band	Model	Precision	Recall	F1	Found	Missed
High 8 cats 176 positives	Transformer	0.404	0.841	0.546	148	28
	TF-IDF	0.540	0.648	0.589	114	62
	AND ensemble	0.585	0.625	0.604	110	66
	OR ensemble	0.391	0.864	0.538	152	24
Medium 22 cats 498 positives	Transformer	0.425	0.871	0.571	434	64
	TF-IDF	0.621	0.723	0.668	360	138
	AND ensemble	0.647	0.673	0.659	335	163
	OR ensemble	0.423	0.922	0.580	459	39
Low 11 cats 674 positives	Transformer	0.813	0.953	0.877	642	32
	TF-IDF	0.909	0.917	0.913	618	56
	AND ensemble	0.924	0.901	0.912	607	67
	OR ensemble	0.803	0.969	0.878	653	21

Note. Threshold 0.5. “Missed” counts gold clauses of that band the model failed to report.

The headline configuration is the worst one for High-risk clauses

Read down the High band. The AND ensemble (the configuration this project reports as its headline, on the strength of aggregate micro-F1) **misses 37.5% of High-risk clauses.** The transformer alone, the component we have spent two sections establishing is the weaker model, misses **15.9%**. The permissive OR rule, which finishes *last* on aggregate micro-F1 (0.696), misses only **13.6%**.

In absolute terms, on 176 High-risk gold clauses in 102 contracts: the ensemble surfaces 110 and loses 66; OR surfaces 152 and loses 24. Choosing the ensemble

over OR on the basis of micro-F1 costs a signer 42 High-risk clauses (uncapped liabilities, IP assignments, non-competes) in exchange for a leaderboard position that Section 5.4.1 already showed is a statistical tie.

The mechanism is the one identified in Section 5.2.4 and it compounds here. High-risk categories are disproportionately rare (*Most Favored Nation*, *Irrevocable/Perpetual License*, *Minimum Commitment* all sit in the sparse tail), the conjunction is hardest to satisfy exactly where training signal is thinnest, and the risk band that most needs recall is the one where requiring agreement destroys it. The Low band shows the inverse: 674 of the 1,348 gold clauses are Low-risk (dates, party names, governing law) and every model scores near 0.9 there. **Aggregate micro-F1 is dominated by the clauses that matter least.**

What we conclude, and what we do not

We do not switch the reported configuration on the strength of this table, and it is worth being explicit about why: choosing OR *after* seeing which rule scores best on the High band of the test set is test-set selection, exactly the practice Section 4.4.1 says this project avoided. The result stands as a diagnosis, not a retuning.

What it does establish is that **our evaluation metric and our stated purpose were never aligned**, and that the misalignment is large enough to invert the model ranking. A system whose declared aim is to warn a signer before they sign should be selected on High-risk recall, or on a risk-weighted F1, not on an average over 41 categories in which two thirds of the mass is dates and party names. Section 6.5 lists the proper version of this experiment: define the risk-weighted objective first, then select the rule on a validation split, then report on the test set once.

Finally, this is a further argument for taking Section 4.2.4 seriously. Every number in Table 5.10 is computed against *our own* assignment of categories to bands. If a lawyer disagrees about which categories are High, the table changes with it. The risk-weighted evaluation inherits all the unvalidated-taxonomy caveats and adds no independent evidence to them.

5.2.3 What the errors actually look like

The confusion matrix counts 311 false positives and 296 false negatives, and counts alone do not tell a reader whether the model is failing stupidly or reasonably. Table 5.11 shows real cases, drawn from the test contracts: false positives ranked by the model's own confidence, since a system's most confident mistakes are the most revealing ones.

Three patterns are worth naming. **Lexical triggers without the obligation** account for a visible share of false positives: a contract that discusses insurance costs is not a contract with an insurance covenant, and a full-document bag-of-words model cannot tell the difference. **Concept-without-vocabulary** explains much of the rare-category failure: a \$5,000-per-month cancellation fee *is* liquidated damages, and no model with two dozen training examples of the category will discover that. And **annotation disagreement** is a real component of the rare tail: the *Third*

Party Beneficiary case above looks like a labelling artefact rather than a miss, which means the 0.000 F1 scores in Section 5.2.4 are a mixture of model failure and ceiling effects in the labels. We did not attempt to quantify that mixture; doing so would need a re-annotation exercise we are not qualified to run.

The span errors point somewhere more actionable. The degenerate “.” prediction and the confident answer from a window that cannot contain the answer are both symptoms of the same missing piece: the extractor has no way to say *nothing here*. SQuAD v2-style unanswerable training, which supplies exactly that, is the concrete fix and it is listed in future work.

5.2.4 Does the AND rule punish rare categories harder than common ones?

Section 5.4.1 defended the conjunctive ensemble on its error profile: it converts the transformer’s inflated false positives into balanced precision and recall. That defence was made on the aggregate, and an aggregate can hide a trade that falls unevenly. It is worth asking where the AND rule’s precision comes from, because there is an obvious reason to suspect the rare tail specifically. Requiring two models to agree is a stricter bar exactly where each model individually has the least training signal to be confident with, so the categories with the fewest positive examples are the ones where the second model is least likely to concur, and agreement is what the rule demands.

We split the 41 categories by *training* frequency, took the bottom ten (11 to 42 positive training contracts, 70 positive test decisions between them), and compared the AND rule against the transformer alone on that tail and on the remaining 31 categories separately. Table 5.12 gives the result, and it is not the same story in the two halves.

On the common categories the ensemble works as advertised. Against the transformer alone it gains 19.4 points of precision for 12.1 points of recall, and F1 rises from 0.716 to 0.792. That is a good trade, and it is where the ensemble’s entire aggregate advantage is generated.

On the rare tail it is not a free lunch: it is a straight swap at no gain. The AND rule buys 32.5 points of precision by giving up 25.7 points of recall, and the F1 lands at 0.343: identical, to three decimals, to the transformer alone. Every point of precision on the rare tail is paid for with recall, and nothing is left over. In absolute terms the transformer alone finds 35 of the 70 rare positives; requiring agreement cuts that to 17. Notably, the plain TF-IDF baseline is the best of the three on this tail (F1 0.353), which is a further reminder of how little the transformer contributes where training signal is thin.

The per-category detail is starker than the micro-average. On five of the ten rarest categories the AND rule scores exactly 0.000, and on three of those (*Price Restrictions*, *Unlimited/All-You-Can-Eat License*, *No-Solicit of Customers*) the transformer alone was finding something (F1 0.095, 0.667, 0.286) before the conjunction zeroed it out. The rule does not merely fail to help on the rare tail; on those categories it removes detections that one component had made.

Why this matters for deployment rather than just for the table. The aggregate micro-F1 weights every one of the 4,182 decisions equally, but a contract signer does not. Missing a rare clause is not obviously less costly than missing a common one, and it may be considerably more so: *Most Favored Nation* sits in the rarest ten and is one of our eight High-risk categories, and both models score 0.000 on it. A system whose ensemble rule is tuned, even by a parameter-free rule, toward precision on frequent categories is optimizing the metric rather than the harm. We report the ensemble as our headline configuration because it is what the aggregate protocol selects, but on this evidence a deployment aimed at protecting a signer would be better served by *running the more permissive rule on the High-risk categories and the conjunction elsewhere*, accepting more false alarms exactly where a missed clause costs the most. We have not implemented that split rule: doing so and evaluating it properly is future work (Section 6.5), not something to bolt on after seeing the test set.

5.3 Final Design Adjustments

Two design choices made before training, max-pooling as the aggregation rule and 0.5 as the decision threshold, were never tuned. After the main evaluation both were revisited, to determine whether adjusting them would change the conclusions or the deployed system.

5.3.1 Ablation: was max-pooling the right aggregation rule?

Section 4.2.1 chose max-pooling because it *is* the multiple-instance decision rule [19], a bag is positive if any instance is, and Sections 4.2.1 and 5.4.3 then blamed max-pooling for the transformer’s inflated false positives. Both statements were made without ever trying an alternative, which is not good enough for a claim the project leans on. We therefore scored every window once and compared aggregation functions on *identical* window scores, so the only thing varying is the rule.

Three things come out of this, and one of them corrects an earlier claim of ours.

At the threshold we actually used, max-pooling was not the best choice. Top-3 mean scores 0.7144 against max’s 0.6943 at the 0.5 boundary: a fifth of the gap to the baseline, available for free from a one-line change. Averaging the three highest-scoring windows is a natural softening of the MIL rule: it still asks whether evidence is concentrated somewhere in the document, but requires that evidence to appear in more than one place, which is exactly the guard against a single spurious window that Section 4.2.1 identified as max-pooling’s weakness. Mean-over-all-windows collapses at 0.5 (recall 0.091) for the obvious reason, an average over 35 windows of which one is positive cannot reach 0.5, and its best threshold is correspondingly 0.10.

Once each rule is given its own threshold, max-pooling is the best of the six. At 0.90 it reaches 0.7504, ahead of top-3 mean’s 0.7210. So max-pooling is not a poor aggregation rule; it is a rule whose output distribution is badly matched to a 0.5 decision boundary, which is the same finding Section 5.3.2 reached from the

other direction. The two results are one result: the transformer’s reported deficit is substantially an operating-point problem, not an aggregation problem.

But no aggregation rule reaches the baseline. The best cell in the entire table (max-pooling at a threshold tuned on the test set, which is already a generous upper bound) is 0.7504 against the baseline’s 0.7747. Every alternative rule, at every threshold we swept, remains behind. This matters more than the first two points, because it closes off the last available explanation for RQ1: the windowed transformer does not lose because of its training budget (Section 5.4.3), or its seed (Section 5.4.2), or its aggregation rule, or its threshold. It loses.

A correction to Section 5.4.3. There we wrote that “the ceiling here is the aggregation rule, not the encoder underneath it”, inferring it from the precision/recall shift when training data was doubled. Table 5.13 shows that inference was wrong: changing the aggregation rule (through six variants, each at its best threshold) moves micro-F1 by at most 0.056 and never past the baseline. The aggregation rule is a real but secondary factor. We have left the original sentence in place with this correction attached, on the same principle as elsewhere in this chapter: an inference we drew from indirect evidence and later measured properly is more useful to a reader than a silently amended one.

A caveat on how far this ablation goes. It varies only the function applied to window scores. It does not vary the window geometry (2,000 characters, 1,500 stride), which would require re-windowing and retraining, nor does it try a learned aggregator such as attention pooling over window representations. Both remain untested, and attention pooling in particular is the obvious next thing to try, since it could learn to weight windows by relevance instead of taking a fixed statistic.

5.3.2 The Threshold and What the Confidence Score Means

Every presence decision in this project, and every clause card in Clausify, comes from comparing a probability against 0.5. Section 4.4.1 records that this threshold was never chosen: it is the default decision boundary of a two-class softmax, inherited rather than selected. Two questions follow, and neither had been asked: is 0.5 a sensible operating point, and does the number being compared against it mean anything? Figure 5.4 answers both.

0.5 is a poor operating point for the transformer

Sweeping the threshold post hoc shows the max-pooled transformer reaches micro-F1 **0.750 at a threshold of 0.9**, against **0.694** at the default: a gain of 0.056, which is most of the way to closing its gap with the TF-IDF baseline (0.775). The reason is the one Section 4.2.1 predicted structurally: a maximum over 35 windows is inflated by construction, so the scores pile up near 1 and a boundary drawn at 0.5 admits far too much. Precision at 0.5 is 0.562; at 0.9 it is 0.728.

We are not adopting 0.9. It was found by looking at the test set, and adopting it would convert a held-out evaluation into a tuned one: the precise failure Section 4.4.1 certifies this project against. The honest statement is narrower and more useful: *a*

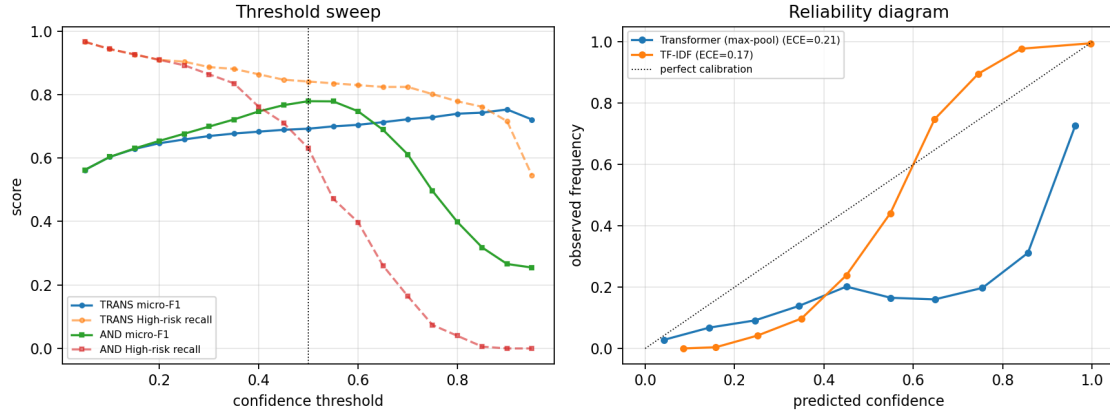


Figure 5.4: Threshold and calibration

meaningful part of the transformer’s reported deficit is an un-chosen threshold rather than an inability to discriminate, and our comparison in Section 5.4.1 therefore compares a tuned-by-construction baseline against an untuned transformer. A fair rematch selects the threshold for each model on a validation split carved from the 408 training contracts, and we did not do that. Section 5.4.3 showed the gap is not a data-budget artefact; this section shows part of it may be a threshold artefact, and that qualification belongs on the finding.

The AND ensemble behaves oppositely: 0.5 is already its optimum (0.776), and raising the threshold collapses it: at 0.7 micro-F1 falls to 0.610 and High-risk recall to 0.159, because requiring two models to *both* clear a high bar is close to requiring unanimity among the confident. The dashed curves in Figure 5.4 make the tension visible: for the transformer, the threshold that maximizes micro-F1 (0.9) drives High-risk recall down from 0.841 to 0.688. There is no single threshold that is simultaneously best for the aggregate metric and for the clauses that matter most.

The confidence bar in the application is not a probability

Clausify displays the presence score to the user as a confidence bar. The reliability diagram tests whether that display means what a user would take it to mean, and it does not.

The max-pooled transformer has an **Expected Calibration Error of 0.201** and a Brier score of 0.194. It is overconfident across the entire range: among predictions in the [0.5, 0.6) bin the clause is actually present **19.5%** of the time, and even in the top bin (scores above 0.9, which the interface renders as a nearly full bar) the clause is present **72.8%** of the time. A user reading “96% confident” is looking at something closer to a 73% chance. Again the cause is max-pooling: taking the maximum over dozens of windows is not a probability-preserving operation, and nothing in training asked it to be.

The TF-IDF baseline is differently miscalibrated (ECE 0.172) but in the safer direction over most of its range: it is *underconfident* in the middle, with the [0.7, 0.8) bin correct 89.6% of the time and the [0.8, 0.9) bin 97.8%.

The practical consequence for the deployed system is direct and we have recorded it

in Section 5.6.2: the presence score the application shows on every card is in units that do not correspond to probability of correctness. Fixing this needs no retraining, Platt scaling or isotonic regression fitted on a validation split would map scores onto probabilities, and it is the cheapest single improvement available to the interface. We did not have a validation split set aside to fit it on, which is a consequence of the protocol gap in Section 4.4.1 rather than an independent oversight.

5.4 Statistical Analysis

The statistical analysis uses a paired cluster bootstrap [18]: the 102 test contracts are resampled with replacement 10,000 times, and every model is re-scored on the same resample, so that within-contract correlation between the 41 decisions is respected. Three further sources of variance that the bootstrap cannot capture are then measured directly: training budget, random seed and the choice of split.

5.4.1 How much of that ranking is real?

Table 5.1 is a leaderboard, and leaderboards invite a reader to treat the ordering itself as a result. On 102 test contracts a margin of 0.776 against 0.775, roughly one decision in a thousand, plainly cannot support that reading, so rather than leave the caveat as prose we measured it.

The natural tool is a **bootstrap confidence interval**, but the resampling unit matters. The 4,182 test decisions are not independent: 41 of them come from the same contract, and a contract that is unusual (unusually long, unusually drafted) moves all 41 together. Resampling individual (contract, category) cells would ignore that structure and produce intervals that are far too narrow. We therefore resample *contracts* with replacement (a cluster bootstrap), drawing 102 contracts from the 102 available, recomputing micro-F1 for both models on each resample, and repeating 10,000 times. Both models are scored on exactly the same resample every time, so the difference is *paired*, which is what allows a difference interval much tighter than the two individual intervals would suggest.

The three comparisons do not come out the same way, and the difference between them is the point.

The ensemble’s win over the baseline is not real. The interval is $[-0.0068, +0.0089]$: it straddles zero, and the ensemble is ahead in only 62.6% of resamples, barely better than a coin flip. Taken individually the two models score 0.776 (95% CI $[0.759, 0.792]$) and 0.775 (95% CI $[0.758, 0.791]$), intervals that almost entirely overlap. On this test set the AND-ensemble and the TF-IDF baseline are *statistically indistinguishable*, and we now state that as a measurement rather than as a hedge. The top two rows of Table 5.1 should be read as a tie.

The transformer’s loss, by contrast, is real. The baseline beats the windowed transformer by 0.0805, with an interval of $[+0.0626, +0.0992]$ that is nowhere near zero, and it wins in every one of the 10,000 resamples. This matters for how the project’s two empirical claims should be weighted, because they are not on equal

footing. The finding we stand behind most firmly, that a full-document lexical model beats a windowed DistilBERT at document-level presence classification, is the one that clears the significance test comfortably. The finding we were already cautious about, that the conjunctive ensemble edges past the baseline, is the one that does not. That our caution happened to point the right way is reassuring, but it was worth checking rather than assuming, and a reader should not have had to take it on trust.

None of this undercuts the case for the ensemble, because that case was never the 0.001 of micro-F1. The AND rule earns its place through the *error profile* it produces rather than its position on the leaderboard: it converts the transformer’s inflated false positives into balanced precision and recall (Table 5.3); Sections 5.4.2 and 5.2.4 examine what that profile costs.

5.4.2 How much of this moves if we only change the random seed?

Section 5.4.1 measured one kind of uncertainty, how much the score depends on *which contracts* landed in the test set, and used it to call the ensemble’s margin a tie. There is a second kind it cannot see. Every number reported so far comes from a single training run per model, and a fine-tuning run is stochastic: weight initialization of the classification head, data shuffling, and the negative-window sample all depend on the seed. If simply reseeding moves the score as much as the effects we are discussing, then a leaderboard built from single runs is reporting noise regardless of how the test set was drawn.

This is easy to check and we had not checked it, so we retrained the presence transformer twice more at the same 24,000-example budget, changing nothing but the seed (42, 43, 44), and re-evaluated each on the same 102 contracts. Table 5.15 reports the spread.

The ensemble’s margin is smaller than its own seed noise. The AND ensemble’s advantage over the baseline, the number Table 5.1 ranks it by, is +0.0014. The standard deviation of that same ensemble across seeds is 0.0022, and the observed range is 0.0042: three times the margin. Reseeding the model moves the score further than the result being claimed for it. This is an entirely independent line of evidence pointing where the cluster bootstrap already pointed: had we trained with seed 44 and stopped, we would have reported a margin of +0.0056 and been just as wrong to make anything of it. A single-run leaderboard separated by one part in a thousand was never measuring what it appeared to measure.

The finding we do stand behind is untouched by seed noise. The baseline beats the transformer by 0.0805, which is 9.5 times the transformer’s seed-level standard deviation of 0.0085. Even the luckiest of the three seeds, at 0.7082, still loses to the deterministic baseline by 0.0665. There is no seed at which the windowed transformer becomes competitive, and combining this with Section 5.4.3, where doubling the training data also failed to close the gap, leaves the architectural explanation as the one standing after both alternatives were tested.

It is worth noticing which model is noisier, because it is not the one a reader might

guess. The transformer alone varies four times as much across seeds ($SD = 0.0085$) as the ensemble containing it (0.0022). Requiring agreement with a deterministic lexical model damps the transformer’s run-to-run variation as well as its false positives: a spurious detection has to survive a second, fixed opinion, and seed-driven fluctuations in what the transformer fires on are largely filtered out. That stabilization is a real and previously unreported property of the conjunctive rule: and, on the evidence of Section 5.2.4, one it buys at the price of rare-category recall. It is a better argument for the ensemble than its position on the leaderboard ever was.

Scope of this check. We reseeded the presence transformer only. The span extractor and the summarizer were each trained twice with the *same* seed (August and October 2026) and reproduced their scores to within 0.001 (token-F1 0.763 and 0.764; ROUGE-L 0.775 both times), which bounds run-to-run noise on this hardware but says nothing about seed variance. That component remains unmeasured for those two models, and we flag it in Section 6.3 rather than assume it is small.

5.4.3 Was the transformer simply undertrained?

There is an objection to Section 5.4.1 that the bootstrap cannot answer, and it goes to the heart of the project’s central empirical claim. The TF-IDF baseline is fitted on all 408 training contracts in full. The window-level transformer was not: to keep the run tractable on a laptop it was trained on a stratified subsample of 24,000 of the 53,662 available window examples (Section 4.4.3). So when the transformer loses by 0.081 micro-F1, we cannot tell from that comparison alone whether max-pooling over windows is genuinely weaker for this task, or whether the transformer simply never saw 55% of its own training set. One reading is an architectural finding; the other is an artefact of our compute budget. They call for very different conclusions, and the difference matters more than any other number in this chapter.

The only way to settle it is to remove the confound, so we retrained the presence transformer on the **full 53,662 window examples**, everything the training split can produce, with every other setting identical: same windowing, same negative sampling, same hyperparameters, same seed, same evaluation. The run took 140.7 minutes on the same laptop, against roughly 50 for the subsampled version. Table 5.16 gives the outcome.

The confound is not the explanation. Increasing the training set by 124% moved micro-F1 by -0.0004 , with an interval of $[-0.0098, +0.0095]$ squarely straddling zero: the two budgets are statistically indistinguishable. And the gap to the baseline is untouched: $+0.0809$ on full data against $+0.0805$ on the subsample, with an interval nowhere near zero and the baseline winning in all 10,000 resamples. Whatever is holding the windowed transformer back, it is not a shortage of training examples, and the finding this project leans on survives the check.

The *shape* of the extra data’s effect is worth a sentence, because it is not simply “nothing happened”. Recall rose from 0.9080 to 0.9266 and precision fell from 0.5620 to 0.5546: more training data made the model fire more readily, not more accurately. That is exactly the failure mode Section 4.2.1 predicted for max-pooling: with a mean of 35 windows per contract, the document-level decision is a maximum over many chances to be wrong, and a better-trained window scorer that is still imperfect

produces *more* spurious maxima, not fewer. The ceiling here is the aggregation rule, not the encoder underneath it.¹

One secondary effect deserves reporting even though it does not change our conclusion. The AND-ensemble built on the full-data transformer scores 0.7820, and its margin over the baseline is +0.0073 with a 95% interval of $[-0.0000, +0.0140]$: it wins in 97.4% of resamples but the interval still touches zero. So the ensemble’s advantage moves from “clearly a tie” at the smaller budget to “borderline” at the larger one. We are not going to claim it: an interval whose lower bound sits on zero does not clear the bar we set for ourselves in Section 5.4.1, and we are not going to lower that bar because the number moved our way. What we will say is that the ensemble’s balanced error profile survives the extra data intact.

Which model the rest of this chapter uses. Every other result we report (the confusion matrix below, the per-category analysis, the end-to-end evaluation of Section 5.2.1, and the deployed Clausify checkpoint) comes from the 24,000-example model. The full-data run is a control experiment, run to test one specific alternative explanation, and we report it as one rather than swapping in the marginally better number after the fact. Since the two are statistically indistinguishable, nothing downstream would change if we had.

5.4.4 A third source of variance: the split itself

Sections 5.4.3 and 5.4.2 tested two things that could move our numbers: the training budget and the model seed. Both left the conclusions standing. There is a third source neither touches, and it is arguably the largest: **the split**. Every result in this project comes from one contract-level 80/20 partition drawn with seed 42. With only 510 contracts, a different draw puts different rare categories in the test set and could move the scores on its own. Model-seed variance is not a proxy for this: reseeding training holds the test set fixed, so it cannot see split effects at all.

Retraining the transformer on twenty splits is not affordable, but the TF-IDF baseline refits in under a minute, and since it is the reference point every comparison in this project is made against, its split-to-split spread bounds how much the comparison itself could move. We drew **20 independent contract-level 80/20 splits** and refitted the baseline on each.

Three readings follow. First, **split variance is the biggest of the three uncertainties we have measured**: SD 0.0063 against 0.0022 for model seed and a bootstrap interval half-width of about 0.008. It is roughly four and a half times the ensemble’s +0.0014 margin, which is now the third independent reason to treat that margin as noise.

Second, **our split is not a lucky one**. At 0.7747 it sits below the 20-split mean of 0.7829, in the lower part of the distribution, so if anything this project reports the baseline at a slightly unfavourable draw. We note this because the alternative would have been worth worrying about: a headline built on a split that happened

¹This inference turned out to be only partly right, and Section 5.3.1 corrects it: swapping max-pooling for five alternatives moves micro-F1 by at most 0.056 and never past the baseline, so the aggregation rule is a secondary factor rather than the ceiling.

to flatter the baseline would have made the transformer’s defeat look worse than it is.

Third, **the finding we rely on is far outside this variation**. The baseline’s lead over the transformer is 0.0805: about thirteen split-level standard deviations. No plausible redraw of the split turns that around.

What we cannot say is anything about *per-category* stability across splits, which is where the instability would really bite: a rare category with two test positives on one draw may have eight on another, and its F1 is meaningless in both cases. Repeated splits or cross-validation for the full pipeline remains future work, and Section 6.4 lists it as a limitation rather than an answered question.

5.5 Comparisons and Relationships

Table 5.18 pulls together the held-out results of all three models. Every number was produced by the standalone test notebook running on the saved 80/20-trained models.

Relationship to existing work. Table 2.1 in Chapter 2 compares this project with prior work. The original CUAD study reports span extraction with DeBERTa-v3-XL under a full-document precision-at-recall protocol; our distilled backbones are roughly an order of magnitude smaller and our span scores are measured within answer-bearing windows, so the absolute numbers are not directly comparable and we do not present them as such. What the comparison does show is a difference in scope: prior work stops at locating clauses, while this project adds presence decisions, a category-level risk layer and plain-language output, and evaluates the complete pipeline end to end.

Relationships between the approaches. Across the presence models, the lexical baseline and the windowed transformer are complementary, the baseline is stronger on keyword-heavy categories and the transformer on semantically subtle ones, which is why the conjunctive ensemble matches the baseline while balancing precision and recall. Across stages, the component scores do not compose: errors in window selection by the presence stage propagate into span extraction, which is why the end-to-end rate is far below the product of the component scores.

5.6 Discussions

Pulling the chapter together, the results fall into three groups, and it matters which is which.

Findings that survived every check we could run.

1. For document-level *presence*, a full-document lexical baseline beats a windowed DistilBERT by +0.081 micro-F1. We tested the two obvious alternative explanations and neither held: training on all 53,662 windows instead of 24,000 changed the score by -0.0004 (Section 5.4.3), and across three seeds

the gap is 9.5 standard deviations wide (Section 5.4.2). More data made the transformer fire *more* rather than better, which is what max-pooling over 35 windows predicts. The limitation is the aggregation rule, not the encoder or the budget.

2. For *span extraction*, the transformer has no substitute, a bag-of-words model cannot point at character offsets, and it performs well in distilled form *when handed the right window*: token-F1 0.763, 95% CI [0.740, 0.782].
3. For *summarization*, a 41-sentence template seed turns the task into classification. The fine-tuned model reproduces a template verbatim on 100% of test clauses, and against clause-specific references it scores no better than the template alone (Section 5.1.3). A high automatic metric should not be believed until the outputs have been read.

A claim we withdrew. The AND-ensemble’s first place in Table 5.1 is not a result. Its margin over the baseline is +0.0014; the cluster bootstrap puts that inside $[-0.0068, +0.0089]$ (Section 5.4.1), and the seed-to-seed standard deviation of the ensemble itself is 0.0022: larger than the margin, with a range three times larger. Two independent sources of uncertainty agree that the top two rows of that table are a tie. What the ensemble does earn is a balanced error profile and a fourfold reduction in seed variance relative to the transformer alone.

Costs we did not see until we looked for them. Two results in this chapter came from questions we had not thought to ask, and both are unflattering.

- The conjunctive rule is not uniformly good. On the ten rarest categories it nets *zero* F1 gain over the transformer alone, buying 32.5 points of precision with 25.7 points of recall and cutting detections from 35 of 70 to 17 of 70 (Section 5.2.4). Its entire aggregate benefit is generated on common categories, and one of the categories it zeroes out is High-risk.
- The stage-wise scores do not compose. Only 21.7% of gold clauses survive the full pipeline, against roughly 64% predicted by multiplying the component numbers (Section 5.2.1), because the presence model’s argmax window is the wrong window 31.7% of the time and the span model was trained on re-centred windows it never sees at inference.

The through-line is methodological rather than architectural. Every number in this chapter that we originally trusted least (the ensemble’s rank, the summarizer’s ROUGE, the span model’s token-F1) turned out to be measuring something narrower than it appeared to, and in each case the way to find that out was to measure the thing the metric was standing in for: resample the contracts, change the seed, vary the reference set, run the stages in series. None of those checks needed a bigger model or a larger dataset. They needed the question to be asked.

5.6.1 Usability

No usability evaluation was carried out. The application’s central claim, that grouping clauses by risk helps a non-lawyer act on a contract, is therefore untested, and Section 6.3 lists it under construct validity rather than among the results.

5.6.2 Honest Limitations of the Demo

By default the app detects clauses with the transformer presence model alone; the AND-ensemble is offered as a setting. As Section 5.4.1 showed, the transformer alone over-detects. Two measured qualifications explain why it is nonetheless the default.

The confidence score is not calibrated in the units a user will read it in. Section 5.3.2 shows the transformer’s max-pooled score has an Expected Calibration Error of 0.201 and is overconfident throughout: among detections scoring above 0.9, a nearly full confidence bar, the clause is actually present about 73% of the time, and among those in the 0.5–0.6 band, under 20%. The application therefore uses the fixed 0.5 threshold of every reported evaluation (an earlier version exposed it as a slider, which invited reading the score as a probability) and labels every bar as a model score, not a probability. Fitting a calibration map on a validation split would fix the underlying problem without retraining.

Running the transformer alone is, unusually, the better choice for High-risk clauses. The project reports the AND ensemble as its headline configuration, so a reader might take the demo’s use of the transformer alone as a compromise forced by packaging. Section 5.2.2 shows it is not a compromise at all where it matters most: the ensemble misses 37.5% of High-risk clauses while the transformer alone misses 15.9%. The demo’s configuration is worse on aggregate micro-F1 and better at surfacing the clauses a signer would most regret missing. It began as an accident of packaging, made before we had run the risk-weighted evaluation; it is now the deliberate default. The ensemble remains available as an option, with one further caveat measured on the application itself: on short contracts its TF-IDF half, trained on full-length SEC filings, vetoes most detections: on the 4,200-character demonstration contract it reports no High-risk clause at all, where the transformer alone reports seven.

The 30-window cap silently truncates long contracts. To bound latency the app scores at most 30 windows, so any contract beyond 45,500 characters is only partly read, and clauses past that point cannot be detected at all. None of the Chapter 5 numbers reflect this, since they score full documents. This is not an edge case: **38.4% of CUAD contracts (196 of 510) exceed 45,500 characters**, and the longest test contract is 289,615 characters, of which the app reads the first 16%. Table 4.6 shows the symptom: the 75th-percentile and longest contracts take the same time despite differing fourfold in length, because both are truncated to 30 windows. Span boundaries inherit the distilled backbone’s approximate edges, and Section 5.2.1 shows this understates the problem: when the extraction window is chosen by the presence model rather than handed over, the quoted text overlaps the real clause in only about a fifth of cases overall. The practical consequence for a user of this demo is worth stating plainly, and the application should state it too: **the risk badge and the category name are much more reliable than the quoted sentence beneath them.** A card should be read as “this contract appears to contain a clause of this type, look here” rather than as “this is the clause”. And the risk levels are per-category, not per-instance: the taxonomy flags that a *Cap on Liability* clause deserves attention, but it does not read whether this particular cap is generous or punishing. All of this is now stated in the notice the application shows

above every analysis. What the demonstration does establish is that both models generalize to contracts they have never seen, and that the output can be presented in a form a layperson can act on.

Table 5.11: Representative errors

Category	Scores	Evidence
False positives: reported, but the category is not annotated in this contract		
Insurance	0.82 / 0.97	“...a fixed amount of [*] of gross sales to cover freight, postage, insurance costs on shipments to such Third Party, packing costs...” <i>A royalty-deduction clause that mentions insurance as a cost item. The word is present; the obligation is not.</i>
Governing Law	0.81 / 0.96	“...there shall arise the need to decide the question by what municipal or national law this Agreement ...is governed , the following facts shall be excluded...” <i>A clause about how to determine the governing law, rather than a clause stating it. Arguably a boundary case in the annotation.</i>
Parties	1.00 / 0.99	An addendum whose opening window reads like a contract header. <i>Both models fire on document-structural cues; the annotation places Parties elsewhere.</i>
False negatives: annotated as present, both models scored it low		
Liquidated Damages	0.21 / 0.02	“...agrees to pay a cancellation fee of Five Thousand Dollars (\$5,000) per month multiplied by the number of months remaining...” <i>A textbook liquidated-damages term that never uses the phrase. The lexical model has no signal; the transformer, trained on 24 positive examples of this category, has not learned the concept.</i>
Most Favored Nation	0.22 / 0.02	“...Sponsor shall have the right to approve or disapprove any additional sponsor...unless another...” <i>Semantically an MFN-style protection, expressed without any of the category’s vocabulary. This category has 21 positive training contracts and scores 0.000 overall.</i>
Third Party Beneficiary	0.26 / 0.02	“Notwithstanding the foregoing, UTK may assign this Agreement or any portion of its Compensation ...to its subsidiaries.” <i>This reads like an assignment clause, not a third-party-beneficiary clause. Some of the rare-category “failures” are disagreements with the annotation rather than model errors.</i>
Span errors: category correctly detected, extracted text wrong (Section 5.2.1)		
Covenant Not To Sue	F1 0.00	Window <i>does</i> contain the clause; the model returns “.”: a degenerate single-token span. <i>A start/end pair collapsing onto punctuation, which the argmax decoding has no constraint against.</i>
Effective Date	F1 0.06	Window <i>does not</i> contain the gold clause; the model confidently returns “7th day of september, 1999”. <i>When Stage 2A supplies the wrong window, Stage 2B answers anyway: there is no abstention mechanism.</i>
Insurance	F1 0.20	Gold is the insurance obligation; the model returns the neighbouring indemnification clause. <i>Topically adjacent, legally distinct.</i>

Note. Text abridged. Scores are the TF-IDF and transformer probabilities.

Table 5.12: Rare vs. common categories

Group	Model	Precision	Recall	F1	TP	FN
Rarest 10 (70 test positives)	Transformer	0.261	0.500	0.343	35	35
	TF-IDF	0.429	0.300	0.353	21	49
	AND ensemble	0.586	0.243	0.343	17	53
Other 31 (1,278 test positives)	Transformer	0.582	0.930	0.716	1,189	89
	TF-IDF	0.753	0.838	0.793	1,071	207
	AND ensemble	0.776	0.810	0.792	1,035	243

Note. Micro-averaged within each group.

Table 5.13: Aggregation ablation

Aggregation	F1@0.5	P@0.5	R@0.5	Best F1	at threshold
Max (used in this project)	0.6943	0.562	0.908	0.7504	0.90
Top-3 mean	0.7144	0.667	0.769	0.7210	0.40
Top-5 mean	0.6661	0.719	0.620	0.7192	0.30
Mean over all windows	0.1664	0.946	0.091	0.7114	0.10
Noisy-OR	0.5897	0.423	0.973	0.7022	0.95
Second-highest window	0.6832	0.662	0.705	0.7016	0.30
<i>TF-IDF baseline (no windows)</i>	0.7747	0.742	0.810	–	–

Note. “Best” is each rule’s micro-F1 at its own optimal threshold, found by sweeping the test set: a diagnostic upper bound, not a configuration we adopt.

Table 5.14: Cluster bootstrap results

Comparison	Difference in micro-F1	95% CI	$P(> 0)$
AND-ensemble vs. TF-IDF baseline	+0.0014	[−0.0068, +0.0089]	0.626
TF-IDF baseline vs. transformer	+0.0805	[+0.0626, +0.0992]	1.000
AND-ensemble vs. transformer	+0.0818	[+0.0654, +0.0987]	1.000

Note. 10,000 resamples. The paired difference is what settles each comparison; $P(> 0)$ is the fraction of resamples in which the first model wins.

Table 5.15: Seed variance

Model	Seed 42	Seed 43	Seed 44	Mean	SD	Range
Transformer (max-pool)	0.6943	0.6928	0.7082	0.6984	0.0085	0.0154
AND ensemble	0.7761	0.7774	0.7803	0.7780	0.0022	0.0042
TF-IDF baseline	0.7747 (deterministic)			0.7747	–	–

Note. All three runs use the 24,000-example budget and differ only in seed; the TF-IDF baseline is deterministic and appears once.

Table 5.16: Training-budget control

Model	Training windows	micro-F1	Precision	Recall
Transformer (max-pool)	24,000 (44.7%)	0.6943	0.5620	0.9080
Transformer (max-pool)	53,662 (100%)	0.6939	0.5546	0.9266
TF-IDF baseline	all 408 contracts	0.7747	0.7424	0.8101
<i>Paired differences (95% CI over 10,000 contract resamples)</i>				
Full-data transformer – subsampled transformer		−0.0004	[−0.0098, +0.0095]	
TF-IDF baseline – full-data transformer		+0.0809	[+0.0649, +0.0980]	

Note. Evaluated identically on the 102 held-out contracts; differences use the paired cluster bootstrap of Section 5.4.1.

Table 5.17: Split variance

	Mean	SD	Min	Max	Range
micro-F1 over 20 splits	0.7829	0.0063	0.7702	0.7918	0.0216
<i>The split used throughout this project (seed 42): 0.7747</i>					

Table 5.18: Final test results

Stage	Model	Metric	Score
2A Presence	AND-ensemble (TF-IDF + DistilBERT)	micro-F1	0.776
2A Presence	AND-ensemble	Accuracy	85.49%
2A Presence	AND-ensemble	Precision / Recall	77.18% / 78.04%
2A Presence	AND-ensemble	Specificity	89.03%
2B Span	DistilBERT-QA	token-F1 (given the window)	0.763
2B Span	DistilBERT-QA	token-F1 (window chosen by 2A)	0.386
2B Span	DistilBERT-QA	Exact Match	35.5%
1 Summarizer	FLAN-T5-small	ROUGE-L (41 templates)	0.775
1 Summarizer	FLAN-T5-small	ROUGE-L (clause-specific)	0.116
Full pipeline	all three stages	gold clauses surviving	21.7%

Chapter 6

Conclusion

6.1 Summary of Findings

We set out to turn CUAD’s clause-extraction benchmark into something a contract signer could use: a system that finds the clauses, says how risky they are, and explains them in plain English. On a strict contract-level 80/20 split the delivered pipeline reaches 85.5% accuracy and micro-F1 0.776 for clause presence and token-F1 0.763 for extraction given the right window, and it runs live inside *Clausify*.

Those are component scores, and the single most useful thing this project can say is that component scores are not the system. Answering our five research questions (Section 1.3.1) produced three negative results, and they are the contribution:

- **The windowed transformer does not beat the lexical baseline** (RQ1), and it does not lose for any of the reasons we could have blamed. Not the training budget: the full 53,662-window run moves micro-F1 by -0.0004 . Not the seed: three runs put the gap at 9.5 standard deviations. Not the aggregation rule: six alternatives, each at its own best threshold, all stay behind. The threshold accounts for part of it, an un-chosen 0.5 costs 0.056 micro-F1, and that qualification belongs on the finding, but even the best cell we could construct remains below the baseline.
- **The summarizer does not summarize** (RQ3). It emits a category template verbatim on every input, and a constant lookup table matches its score against clause-specific references. This is a proof-of-concept classification-to-template component and we describe it that way everywhere.
- **The pipeline loses most of what its parts find** (RQ4). 21.7% end-to-end survival against $\approx 64\%$ predicted, localized to a window-selection failure and a train/inference mismatch: both fixable without a larger model.

RQ5 produced the result we least expected and consider most important for anyone building a system like this. Our evaluation metric and our stated purpose were never aligned: the AND-ensemble wins on micro-F1 while missing 37.5% of High-risk clauses, and the OR rule that finishes last on micro-F1 misses 13.6%. Two thirds of the aggregate’s mass is Low-risk categories: dates, party names, governing

law. A system built to warn a signer before they sign was being selected on a metric dominated by the clauses that matter least.

Five lessons seem likely to generalize.

First, a strong classical baseline deserves to be taken seriously, and beating it must be demonstrated rather than assumed, and when it is not beaten, the alternative explanations should be tested one at a time rather than conceded or hand-waved.

Second, a margin should be compared against its own noise before it is reported. We measured three sources: bootstrap over contracts (≈ 0.008 half-width), model seed (SD 0.0022) and dataset split (SD 0.0063). All three exceed the $+0.0014$ we had ranked a leaderboard by.

Third, an automatic metric measures its reference set, not the task. ROUGE-L 0.775 became 0.116 when the references changed and nothing else did.

Fourth, evaluate the pipeline, not the parts, and evaluate it against the harm it is supposed to prevent, not against an average that treats a governing-law clause and an uncapped indemnity as equally important.

Fifth, feasibility ceilings are cheap to measure and expensive to assume. A one-minute retrieval check (64%) killed an architecture before it cost training time; the same discipline applied to sparse attention corrected *us*, since a Longformer training step does fit in this laptop’s memory at batch size 1, contrary to what we had asserted.

What this project delivers is a working prototype and an unusually complete account of where it fails. Two gaps are large enough that we would not present the system as more than a prototype until they are closed: **no one with legal training has reviewed the risk taxonomy**, which is the main thing this work adds over CUAD, and **no user has ever been observed using the application**, so its central claim, that a risk-prioritized report helps a non-lawyer, remains untested. Both are listed first in the next section, and neither needs more compute.

6.2 Contributions to the Field

The main contributions of this work are:

1. **A prototype category-level risk taxonomy for CUAD.** A High/Medium/Low level with a written justification for each of the 41 categories, scored from the weaker party’s perspective (8 High, 22 Medium, 11 Low), together with explicit assignment criteria (Section 4.2.4). We present it as an *initial framework intended for expert validation*, not as a validated legal-risk model: it was authored by the three of us, none of whom has legal training, and as of this writing no legal professional has reviewed it (Section 4.2.4). It attaches risk to a *category*, not to the wording of a particular clause, and we call it category-level risk prioritization throughout for that reason.
2. **An empirical result on long-document presence classification, with its alternative explanations tested.** A full-document TF-IDF baseline

(micro-F1 0.775) turned out to be hard to beat. Our windowed DistilBERT loses to it (0.694), and we checked the two obvious excuses: retraining on all 53,662 window examples instead of 24,000 changed the score by -0.0004 , and across three random seeds the gap is 9.5 standard deviations wide. An AND-ensemble nominally finishes first (0.776), but both a paired cluster bootstrap and the ensemble’s own seed variance show that margin is indistinguishable from zero, so we report the ensemble for its error profile rather than its rank.

3. **An end-to-end measurement of error compounding.** Evaluating the three stages in series rather than in isolation shows that only 21.7% of gold clauses survive the whole pipeline, against the roughly 64% the component scores predict, and localizes the loss to a window-selection failure and a train/inference mismatch: both fixable without a larger model.
4. **A demonstration that a small template set turns summarization into classification.** Evaluating the same 150 held-out clauses against our 41 category templates and against clause-specific references drops ROUGE-L from 0.775 to 0.116, and the fine-tuned model is shown to emit a template verbatim on 100% of inputs.
5. **A methodological guardrail: measure the ceiling before training.** We show that question-guided retrieval windowing caps attainable recall at 64%, and we used that one measurement to reject the whole retrieval approach before spending any training time on it.
6. **A reproducible, laptop-scale pipeline.** All three models train on a single Apple-Silicon laptop. The split, the trained weights, and a standalone evaluation notebook are all saved, so every number in this project can be regenerated from disk.
7. **Clausify**, a prototype web application that runs the trained models live and presents the results grouped by risk. It is a demonstration of the pipeline, not an evaluated tool: no usability study was conducted, and Section 5.6.2 records what it does not do well.

6.3 Threats to Validity

The results in Chapter 5 rest on choices that could, if wrong, change the conclusions. We set them out under the four standard headings, distinguishing throughout between threats we *measured* and threats we merely *acknowledge*.

6.3.1 Internal validity: could our own procedure have produced these numbers?

Hyperparameters were never tuned (Section 4.4.1). This removes the usual threat, no setting was chosen by scoring variants on the test set, but introduces its mirror image: the transformer competes at conventional defaults against a baseline that has essentially no hyperparameters to get wrong. Section 5.3.2 shows this is not hypothetical: the un-chosen 0.5 threshold costs the transformer 0.056 micro-F1,

so part of the headline gap is a threshold artefact rather than a modelling result. *This is the threat we consider most serious, and it qualifies our central finding.*

The summarizer’s exploratory run monitored loss on test-split clauses (Section 4.4.1). No checkpoint was selected from it and the deployed model was rebuilt without any evaluation set, so no reported score depends on it, but the practice was wrong and the “validation” loss it printed should be discounted. Figure 4.11 now shows the October 2026 re-training, which used no evaluation set.

Design decisions were made after inspecting training data, which is legitimate, but the boundary needs care. The window/stride geometry follows from the median clause length measured on training contracts; the retrieval ceiling of 64% was measured across the whole corpus, test contracts included. That measurement rejected an architecture rather than selecting a hyperparameter, and it was made before any model existed, so we judge the leakage risk immaterial, but it is not zero and we would draw the split first if repeating the work.

No seed variation for span and summarizer. Only the presence model was reseeded (Section 5.4.2). The span and summarizer were re-trained once with the same seed, reproducing their scores to within 0.001, so their seed-variance component remains unmeasured.

6.3.2 Construct validity: do our metrics measure what we claim?

ROUGE-L does not measure summarization quality here, and we can prove it. Against 41 self-authored templates it reports 0.775; against clause-specific references, 0.116, and a constant lookup table scores the same 0.115 (Section 5.1.3). We report both numbers precisely because the first one, alone, would be a construct-validity failure. What we still lack is any measure of *faithfulness* or readability (Section 2.2.4), so even the 0.116 does not establish that the output preserves legal meaning.

Micro-F1 does not measure user harm. Section 5.2.2 shows the two can point in opposite directions: the configuration that wins on micro-F1 misses 37.5% of High-risk clauses while a configuration that ranks last misses 13.6%. Two thirds of the aggregate’s mass is Low-risk categories: dates, party names, governing law. Any claim of the form “this system is 77.6% good” inherits this problem.

Category-level risk is not clause-level risk (Section 4.2.4). Our labels say a *category* typically deserves attention; they say nothing about whether a particular cap is generous or punitive. Calling the output a risk assessment would overstate the construct, which is why we call it prioritization.

The risk bands are our own construct. The risk-weighted evaluation is computed against our unvalidated assignment of categories to levels (Section 4.2.4); if a lawyer reassigns categories, those numbers move with them.

Token-F1 against a single annotated span assumes the boundary is well defined. Section 5.1.2 notes that legal span boundaries are genuinely debatable, and Section 5.2.3 shows at least one “false negative” that looks like an annotation

disagreement rather than a model error.

6.3.3 External validity, where do these results transfer?

Every number comes from CUAD: **510 US commercial contracts drawn from SEC filings, in English**. We have not evaluated a single contract from outside that corpus. Specifically we have no evidence about: other drafting traditions or jurisdictions, including **Bangladeshi contracts**, which is the deployment context motivating this work; consumer or employment agreements, which differ in structure and in who the weaker party is; non-English contracts; contracts shorter or far longer than CUAD’s distribution; and contemporary machine-drafted agreements. The risk taxonomy carries an additional jurisdictional threat: a non-compete’s enforceability varies by jurisdiction, so a level that is right in one place may be wrong in another. CUAD’s SEC provenance also biases toward large-corporate agreements between represented parties, which is close to the opposite of the small-business and individual signers the system is aimed at.

6.3.4 Statistical conclusion validity: are the intervals trustworthy?

Three sources of variance were measured, and they disagree in magnitude: the cluster bootstrap over contracts gives an interval half-width of about 0.008 (Section 5.4.1); model-seed SD is 0.0022 over three runs (Section 5.4.2); dataset-split SD is 0.0063 over twenty splits (Section 5.4.4). All three exceed the ensemble’s +0.0014 margin, which is why we call it a tie with some confidence. Split variance is the largest, and it is the one we could measure only for the baseline.

Three seeds is a small sample from which to estimate a standard deviation; the interval around 0.0022 is itself wide.

Rare-category statistics are not meaningful individually. Five categories have at most seven test positives. Their per-category F1 values are reported for completeness, not as estimates, and no claim in this project rests on any of them individually.

One split, one corpus. The bootstrap resamples contracts within our test set; it cannot represent variation across corpora, and no interval in this project should be read as covering that.

6.4 Limitations

Section 6.3 analysed the threats to each kind of validity. This section lists the limitations of the artefact itself: what the system does not do, as distinct from what could undermine the measurements. Both are stated where they arise in the text; these are pointers.

No usability evaluation (Section 5.6.1). Nobody outside the three of us has used

Clausify. The project argues that a risk-prioritized report is more useful to a non-lawyer than a list of clauses, and offers no evidence for it: no task-completion measurement, no comprehension check, no trust or over-reliance measurement. Given that Section 5.3.2 shows the confidence display is miscalibrated and Section 5.1.3 shows the plain-English sentence is category-generic, over-reliance is the specific thing a usability study would need to look for.

Backbone scale. We downsized the backbones (DistilBERT, FLAN-T5-small) to fit a laptop compute budget, so our absolute scores understate what the same method would achieve with larger backbones on CUDA hardware.

Single-seed and single-budget training, partly addressed. Section 5.4.3 reports what happened when the presence transformer was retrained on all 53,662 window examples instead of the 24,000-example subsample, and Section 5.4.2 reports the spread across three training seeds. The other two models, span and summarizer, have each been trained twice with the same seed (reproducing to within 0.001) but never reseeded, at a subsampled budget (8,000 of 11,156 and 4,000 of 11,156), and we have not measured their seed variance at all. Their reported scores should be read with that in mind.

The rare tail is close to unlearnable at this dataset size, and Section 5.2.4 shows the conjunctive ensemble makes it worse rather than better: on the ten rarest categories the AND rule nets no F1 gain over the transformer alone, buying precision entirely at the cost of recall, and it zeroes out three categories that the transformer alone was partly finding. Since one of those rare categories is High-risk in our taxonomy, this is a limitation of the deployed system and not only of the leaderboard.

Span evaluation scope. Section 5.1.2 measures extraction within answer-bearing windows, not the heavier full-document precision-at-recall benchmark. Section 5.2.1 closes part of that gap by measuring what survives the real pipeline, but it is not the same protocol as the CUAD paper’s and we do not present it as comparable.

The summarizer does not summarize. Section 5.1.3 establishes this quantitatively rather than as a caveat: the fine-tuned model emits one of 41 pre-written templates verbatim on every input, and against clause-specific references it scores no better than the bare template with no model at all. It is a classification-to-template component and should be described as one.

The risk taxonomy has not been externally validated. It was written by three computer science students, and as of this writing no legal professional has reviewed it (Section 4.2.4). It is also per-category rather than per-instance: it flags a category’s typical danger, not the severity of the specific wording in front of the user.

One dataset, one jurisdiction. Every number in this project comes from CUAD: 510 US commercial contracts drawn from SEC filings. We have not tested the pipeline on a single contract from outside that corpus, so we have no evidence about how it behaves on other drafting conventions, and none at all about the Bangladeshi deployment context that motivated the work.

6.5 Recommendations for Future Work

- **Get the taxonomy reviewed.** The first thing we would do with another week: hand `review/risk_taxonomy_review_sheet.csv` to two or three law students or practitioners, run `review/agreement.py`, and report Cohen’s or Fleiss’ κ and the disputed categories whatever they turn out to be. Half an hour of someone else’s time each; it is the cheapest credibility this project can buy.
- **Observe someone using Clausify.** Five to ten participants, each reviewing a contract with and without the tool, measuring task time, clauses correctly identified, and, most importantly, whether users over-trust the output. The project claims a risk-prioritized report helps a non-lawyer; nothing in it tests that claim.
- **Select on the objective that matters.** Define a risk-weighted criterion (High-risk recall, or an F1 weighting High clauses more heavily), carve a validation split from the 408 training contracts, select the ensemble rule and per-model thresholds on it, and report once on the test set. Section 5.2.2 suggests OR would win; choosing it on the evidence we have would be test-set selection.
- **Calibrate the confidence score.** Platt scaling or isotonic regression on a validation split would make the application’s confidence bar mean what a user reads it to mean (ECE 0.201 at present). No retraining required.
- **Give the span model a way to abstain.** SQuAD v2-style unanswerable examples would stop it answering confidently from windows that cannot contain the clause: one of the two causes of the 21.7% end-to-end rate (Section 5.2.1).
- **Train the span model on the windows it will actually see.** Varying the answer offset during training, instead of re-centring every example 150 characters in, addresses the other cause.
- **Evaluate summaries for faithfulness, not overlap.** BERTScore, an entailment-based consistency check, a readability statistic, and a small human rating of whether legal meaning survives (Section 2.2.4).
- **Attention pooling over windows.** The ablation in Section 5.3.1 varied only fixed statistics; a learned aggregator that weights windows by relevance is the untested option most likely to help.
- **Cross-validation rather than one split.** Dataset-split SD is the largest uncertainty we measured (Section 5.4.4) and we could only measure it for the baseline.
- **A genuinely abstractive summarizer.** Grow the 41-template seed into a clause-diverse target set and re-fine-tune. The 150-clause pilot in Section 5.1.3 is a working prototype of the annotation protocol and of the evaluation that would detect success; scaling it to a few thousand clauses is the actual path from template classification to simplification.
- **Risk-aware ensemble rules.** Section 5.2.4 suggests, but does not test, applying the permissive (OR) rule on High-risk categories and the conjunction

elsewhere, so that recall is protected exactly where a missed clause costs the signer most. This needs its own held-out evaluation: choosing the rule after seeing the test set would invalidate it.

- **Dense retrieval.** Our 64% retrieval ceiling (Section 4.2.1) was measured with TF-IDF query scoring only. A sentence-encoder retriever matches on meaning rather than shared vocabulary and may raise that ceiling enough to make a retrieve-then-read pipeline competitive; the measurement is cheap and we should have run it.
- **A trained sparse-attention baseline.** Section 2.1.2 argues against Longformer on grounds that are partly arithmetic and partly cost, and Section 4.4.7 measures the cost, but nobody has actually trained one on this task. On CUDA hardware, a Longformer-with-windowing baseline is the honest comparison our argument currently substitutes reasoning for.
- **Scale the backbones.** Re-run the unchanged pipeline with DeBERTa-v3-base/large and FLAN-T5-base on CUDA hardware to close the gap to the published CUAD baselines.
- **Full-document span benchmark.** Implement the document-stride ranked evaluation so we can report precision at 80% recall and compare with the CUAD paper apples-to-apples.
- **Cross-clause conflict detection.** Bring the rule engine designed in our earlier project phase (for example, *Exclusivity* vs. *Most Favored Nation*) into the deployed application.
- **Instance-level risk.** Move from per-category risk to per-clause severity by reading the extracted span itself: a cap’s amount, a non-compete’s duration.
- **Contracts from outside CUAD.** Even three locally sourced English-language commercial contracts, run through the pipeline and inspected by hand, would tell us more about generalization than any further tuning on CUAD, and it is the first step towards the Bangladeshi deployment this work was aimed at.

Bibliography

- [1] D. Hendrycks, C. Burns, A. Chen, and S. Ball, “CUAD: An expert-annotated NLP dataset for legal contract review,” in *Proc. 35th Conf. Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [2] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proc. EMNLP*, 2016, pp. 2383–2392.
- [3] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” in *5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (NeurIPS)*, 2019.
- [6] C. Raffel, N. Shazeer, A. Roberts, *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [7] H. W. Chung *et al.*, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.
- [8] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out (ACL Workshop)*, 2004, pp. 74–81.
- [9] The Atticus Project, *Contract understanding atticus dataset (CUAD) v1*, 2021. [Online]. Available: <https://www.atticusprojectai.org/cuad>.
- [10] I. Chalkidis, A. Jana, D. Hartung, *et al.*, “LexGLUE: A benchmark dataset for legal language understanding in English,” in *Proc. 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [11] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The muppets straight out of law school,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 2898–2904.
- [12] L. Manor and J. J. Li, “Plain English summarization of contracts,” in *Proc. Natural Legal Language Processing Workshop (NLLP)*, 2019, pp. 1–11.
- [13] R. Flesch, “A new readability yardstick,” *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948.

- [14] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On faithfulness and factuality in abstractive summarization,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 1906–1919.
- [15] W. Kryściński, B. McCann, C. Xiong, and R. Socher, “Evaluating the factual consistency of abstractive text summarization,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 9332–9346.
- [16] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *Int. Conf. Learning Representations (ICLR)*, 2020.
- [17] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020.
- [18] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [19] O. Maron and T. Lozano-Pérez, “A framework for multiple-instance learning,” in *Advances in Neural Information Processing Systems 10 (NIPS)*, 1998, pp. 570–576.

Appendix A

Full Risk Taxonomy

The complete 41-category risk taxonomy (8 High, 22 Medium, 11 Low), scored from the weaker party's perspective, with the one-line reason shown to users in the Clausify application.

Table A.1: Full risk taxonomy

Category	Risk	Reason
Non-Compete	High	Limits your future income and freedom to work.
Uncapped Liability	High	Unlimited damages: no ceiling on what you can owe.
IP Ownership Assignment	High	You lose ownership of a core asset for good.
Exclusivity	High	Locks you to a single source or buyer.
Most Favored Nation	High	You must always give the other side your best terms.
Liquidated Damages	High	Fixed penalties trigger automatically on breach.
Irrevocable or Perpetual License	High	A grant you can never take back.
Minimum Commitment	High	You are forced to buy or deliver a minimum amount.
Renewal Term	Medium	Auto-renewal can extend the contract unexpectedly.
Notice Period to Terminate Renewal	Medium	Miss the window and you are locked in for another term.
Competitive Restriction Exception	Medium	Carve-outs to competitive limits: read carefully.
No-Solicit of Customers	Medium	Restricts approaching the other side's customers.
No-Solicit of Employees	Medium	Restricts hiring the other side's staff.
Non-Disparagement	Medium	Limits what you can publicly say.
Termination for Convenience	Medium	The other side can walk away with notice.

Category	Risk	Reason
ROFR / ROFO / ROFN	Medium	First-refusal rights can constrain your options.
Change of Control	Medium	A sale or merger may trigger termination.
Anti-Assignment	Medium	You cannot transfer the contract without consent.
Revenue/Profit Sharing	Medium	You must share a slice of revenue or profit.
Price Restrictions	Medium	Limits what prices you may set or change.
Volume Restriction	Medium	Caps how much you can sell or distribute.
Joint IP Ownership	Medium	IP is co-owned: control is shared, not sole.
Non-Transferable License	Medium	The licence cannot be passed on.
Affiliate License (Licensor)	Medium	Licensor's affiliates are pulled into the grant.
Unlimited / All-You-Can-Eat License	Medium	Broad, unlimited licence scope: check who benefits.
Source Code Escrow	Medium	Source is held by a third party under conditions.
Post-Termination Services	Medium	Obligations continue after the contract ends.
Audit Rights	Medium	The other side can inspect your records.
Cap on Liability	Medium	Caps recovery: may under-compensate real losses.
Covenant Not to Sue	Medium	You waive the right to sue over listed matters.
Document Name	Low	Identifies the agreement: informational.
Parties	Low	Names who is signing: informational.
Agreement Date	Low	The signing date: informational.
Effective Date	Low	When terms start: informational.
Expiration Date	Low	When the contract ends: informational.
Governing Law	Low	Which jurisdiction's law applies: neutral.
License Grant	Low	You receive usage rights: generally favorable.
Warranty Duration	Low	States how long the warranty protects you.
Insurance	Low	Required coverage: a protection.
Affiliate License (Licensee)	Low	Extends rights to your affiliates: favorable.
Third Party Beneficiary	Low	Names an outside beneficiary: informational.

Appendix B

Reproducibility Checklist

Everything below describes the environment in which every number in this project was produced. The purpose is that a reader with the repository and this appendix can regenerate any table without asking us a question.

Environment

Python	3.13.5
Operating system	macOS 26.5.2, arm64
Hardware	Apple M4, 10 cores (4 performance / 6 efficiency), 16 GB unified memory
Accelerator	Metal Performance Shaders (MPS); no CUDA at any point
torch	2.8.0
transformers	5.12.1
datasets	5.0.0
scikit-learn	1.6.1
numpy	2.1.3
pandas	2.2.3
rouge_score	installed (ROUGE-L is additionally implemented inline; see below)
matplotlib	figures only

Data

Dataset	CUAD v1, Hugging Face <code>theatticusproject/cuad</code>
Files used	<code>CUAD_v1/CUAD_v1.json</code> (contexts, character offsets); <code>CUAD_v1/master_clauses.csv</code> (exploratory analysis only)
Contracts	510; 41 categories; 20,910 (contract, category) rows
Note	The repository's default <code>load_dataset()</code> configuration exposes only the raw PDFs and cannot be trained on (Section 4.3).

Seeds and splits

- Global seed 42 for the split, all three trainings, and every bootstrap.
- The split is *drawn once* from `numpy.random.default_rng(42).permutation`, saved to `artifacts/split.json`, and loaded from disk by every subsequent script, so no script re-derives it.
- Additional presence-model seeds 43 and 44 (Section 5.4.2); dataset-split seeds 0–19 for Section 5.4.4.
- Bootstrap: 10,000 resamples, `default_rng(42)`, resampling unit = contract.

Artifact fingerprints

First 16 hex characters of the SHA-256 of each artifact, so a reader can confirm they hold the same files:

<code>CUAD_v1.json</code>	<code>ed0b77d85bdf4014</code>
<code>artifacts/split.json</code>	<code>177e2ef48dfab9cd</code>
<code>artifacts/baseline.pkl</code>	<code>1c26214794393bdb</code>
<code>outputs/presence_mil/final/model.safetensors</code>	<code>4b1aec3cc43aeea9</code>
<code>outputs/span/final/model.safetensors</code>	<code>f97ba9b172beac42</code>
<code>outputs/summarizer/final/model.safetensors</code>	<code>9fcb6bdf777d033c</code>

Commands, and which table each regenerates

Command	Regenerates
<code>python notebooks/build_artifacts.py</code>	All three checkpoints, the split and the fitted baseline (≈ 85 min)
<code>jupyter run notebooks/test.ipynb</code>	Tables 6.1–6.5 (presence, confusion matrix, span, summarizer)
<code>python analysis/bootstrap_ci.py</code>	Table 6.2 (cluster bootstrap)
<code>python analysis/presence_run.py --tag full53k --max-train 0</code>	Table 5.16 (full-data control, ≈ 140 min train + 50 min eval)
<code>python analysis/presence_run.py --tag s43 --seed 43</code>	Table 5.15 (seed variance; repeat for 44)
<code>python analysis/compare_runs.py</code>	Tables 5.16 and 5.15 contrasts
<code>python analysis/span_ci.py</code>	Table 5.5 (span intervals)
<code>python analysis/rare_tail.py</code>	Table 5.12
<code>python analysis/risk_threshold_calib.py</code>	Table 5.10, Figure 5.4
<code>python analysis/split_variance.py</code>	Table 5.17
<code>python analysis/e2e.py AND</code>	Tables 5.8 and 5.9
<code>python analysis/summ_pilot.py</code>	Table 5.6
<code>python analysis/error_examples.py</code>	Table 5.11
<code>python analysis/lf_one.py longformer 4096 1 train</code>	Table 4.5 (one row per invocation)
<code>python analysis/app_bench.py</code>	Table 4.6
<code>python analysis/verify_numbers.py</code>	Cross-checks all 84 reported figures and 17 derived comparisons in Chapters 5–8 against the analysis artifacts; prints 0 mismatched when the text and the data agree
<code>python review/agreement.py <sheets></code>	Reviewer agreement and Cohen’s/Fleiss’ κ once the taxonomy review sheets are returned
<code>tectonic project_report/main.tex</code>	This project report

Known determinism caveats

- MPS training is not bit-reproducible, even run to run on the same machine and library versions. The October 2026 re-training of every model with identical code and seed reproduced the August scores to within about 0.003 (presence micro-F1 0.6943 $\rightarrow$ 0.6924 for the transformer; span token-F1 0.763 $\rightarrow$ 0.764), not exactly.
- ROUGE-L in `summ_pilot.py` is computed by an inline longest-common-subsequence implementation (matching `test.ipynb`) rather than by the `rouge_score` package, so the two are not interchangeable.
- The retrieval-ceiling measurement of Section 4.2.1 was computed over all 510 contracts, test contracts included (Section 6.3).

Appendix C

Repository and Artifact Layout

```
clausify-p3/ (github.com/afnanmz168/clausify-p3) -- research deliverable
|- project_report/ -- this report (LaTeX, CSE400 template)
| | - main.tex / main.pdf
| | - chapters/ core/ appendix/ bibliography/ images/
|- paper/ -- IEEE-format paper (LaTeX + figures)
|- IEEE_Paper_Clausify.pdf
|- data/
| | - train_80.csv -- 80% split (16,728 rows, 408 contracts)
| | - test_20.csv -- 20% split (4,182 rows, 102 contracts)
|- notebooks/
| | - train.ipynb -- training notebook (EDA -> 3 models)
| | - test.ipynb -- STANDALONE evaluation notebook
| | - build_artifacts.py -- retrains & saves all artifacts
| | - artifacts/ -- split.json, baseline.pkl
| | - outputs/ -- presence, span, summarizer checkpoints
|- retrain/ -- October 2026 re-training & re-testing
| | - README.md -- exact terminal commands
| | - stage2a_* stage2b_* stage1_* -- train/test scripts, logs, results
| | - smoke_test.py, make_figures.py
| | - analysis_oct2026/ -- analyses re-run on the new models
|- analysis/
| | - bootstrap_ci.py -- cluster bootstrap, presence models
| | - span_ci.py -- cluster bootstrap, span extraction
| | - rare_tail.py -- rare-category AND-rule breakdown
| | - e2e.py -- end-to-end pipeline evaluation
| | - summ_pilot.py -- two-reference-set summarizer pilot
| | - presence_run.py -- presence training at any data budget
| | - longformer_bench.py -- sparse-attention cost benchmark
| | - diverse_refs.json -- 150 clause-specific pilot references
|- review/
| | - risk_taxonomy_review_sheet.csv -- face-validity review instrument
| | - agreement.py -- reviewer agreement (kappa)
|- demo_clause.txt -- 18-clause demonstration contract

clausify-p3-app/ (github.com/afnanmz168/clausify-p3-app) -- Clausify application
|- app.py -- Streamlit UI
|- model_utils.py -- inference: all four models, clause mode
|- features.py -- file upload, highlighting, reports
|- calibrate_clause_mode.py -- clause-mode calibration (+ .json)
|- cat_questions.json -- 41 category->question strings
|- sample_contract.txt / test_contract.txt
```

```
| - tests/test_app.py -- 38 end-to-end checks
```